

LAB9_RNA

Luis Gilberto González Joel Antonio Jaquez Fabian Morales

2026-05-08

Conjunto de entrenamiento y prueba

```
## Filas entrenamiento: 48442
```

```
## Filas prueba:          12173
```

Para garantizar comparabilidad con todos los modelos desarrollados a lo largo del curso, se reutilizaron exactamente los mismos conjuntos de entrenamiento y prueba generados desde el Laboratorio 3. Utilizando `set.seed(123)` y una división 80/20, el conjunto de entrenamiento quedó con 48,442 observaciones y el de prueba con 12,173, ambos con las mismas 6 variables que son `price`, `accommodates`, `bedrooms`, `bathrooms`, `number_of_reviews` y `review_scores_rating`, aplicando los mismos filtros de precio menor a \$1,000 y habitaciones menores o iguales a 10. Mantener estas condiciones idénticas es fundamental para que cualquier diferencia en el desempeño de las Redes Neuronales respecto a los algoritmos anteriores sea atribuible al modelo y no a los datos.

Variable de respuesta

```
## P33: 136 | P67: 250
```

```
##
```

```
## Distribución entrenamiento:
```

```
##
```

```
## barata    cara    media
```

```
## 16789 14963 16690
```

```
##
```

```
## Distribución prueba:
```

```
##
```

```
## barata    cara    media
```

```
## 4167 3741 4265
```

```
##
```

```
## Muestra de entrenamiento para RNA: 5000 filas
```

Se definió la variable respuesta categórica utilizando los mismos puntos de corte $P33=136$ y $P67=250$ establecidos desde el Laboratorio 4, garantizando consistencia con todos los modelos anteriores. Las propiedades se clasifican en tres categorías que son barata (precio <136), media (136-250) y cara (>250). La distribución es balanceada en ambos conjuntos en entrenamiento hay 16,789 baratas, 14,963 caras y 16,690 medias, y en prueba 4,167 baratas, 3,741 caras y 4,265 medias, lo cual es favorable para el entrenamiento de las redes neuronales ya que ninguna clase domina sobre las demás. Al igual que con SVM, se utilizará una muestra aleatoria de 5,000 observaciones del conjunto de entrenamiento con `set.seed(123)` para garantizar viabilidad computacional y reproducibilidad.

Modelo 1

```
## === Modelo 1 - nnet (1 capa, 5 neuronas, logística) ===

## a 5-5-3 network with 48 weights
## inputs: accommodates bedrooms bathrooms number_of_reviews review_scores_rating
## output(s): precio_cat
## options were - softmax modelling decay=5e-04
```

El primer modelo de RNA fue construido usando el paquete `nnet` con una arquitectura 5-5-3, 5 variables de entrada, 1 capa oculta con 5 neuronas y 3 neuronas de salida (una por cada categoría de precio). El modelo tiene en total 48 pesos a entrenar, lo que lo convierte en una red relativamente simple. La función de activación utilizada es logística (sigmoide) en la capa oculta, y softmax en la capa de salida, lo cual es la configuración estándar para clasificación multiclase ya que softmax convierte las salidas en probabilidades que suman 1, permitiendo asignar cada propiedad a la categoría más probable. Se utilizó un decay de $5e-4$ como regularización para evitar sobreajuste y un máximo de 200 iteraciones para el entrenamiento.

Modelo 2

```
## === Modelo 2 - caret nnet (1 capa, 10 neuronas, CV 10-fold) ===

## Neural Network
##
## 5000 samples
##    5 predictor
##    3 classes: 'barata', 'cara', 'media'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 4499, 4500, 4500, 4500, 4500, 4500, ...
## Resampling results:
##
##   Accuracy   Kappa
##  0.5866009  0.3788535
##
## Tuning parameter 'size' was held constant at a value of 10
## Tuning
##   parameter 'decay' was held constant at a value of 0.1
```

El segundo modelo de RNA fue construido usando `caret` con `method="nnet"` y validación cruzada de 10 pliegues, lo que lo hace más robusto que el modelo 1 al evaluar su desempeño en diferentes subconjuntos de

los datos. La arquitectura utilizada es 5-10-3 que son 5 variables de entrada, 1 capa oculta con 10 neuronas (el doble que el modelo 1) y 3 neuronas de salida con función softmax para clasificación multiclase. Se usó un decay de 0.1, una regularización más fuerte que el modelo 1 ($5e-4$), para compensar la mayor complejidad de la red y evitar sobreajuste. Con 300 iteraciones máximas el modelo tuvo más oportunidades de converger que el anterior. Los resultados de la validación cruzada muestran un Accuracy de 58.74% y un Kappa de 0.3800, métricas que servirán de referencia para comparar con el modelo 1 en la actividad siguiente.

Predicciones con ambos modelos

```
## ===== Accuracy - Conjunto de Prueba =====

## Modelo 1 (5 neuronas, decay=5e-4): 58.51%

## Modelo 2 (10 neuronas, decay=0.1) : 59.02%
```

Ambos modelos muestran un desempeño similar sobre el conjunto de prueba. El Modelo 2 con 10 neuronas y decay=0.1 obtuvo el mejor Accuracy con 59.02%, superando ligeramente al Modelo 1 con 5 neuronas y decay= $5e-4$ que alcanzó 58.51%. La diferencia de apenas 0.51 puntos porcentuales indica que duplicar el número de neuronas no generó una mejora sustancial, lo que sugiere que la limitación no está en la capacidad de la red sino en las variables disponibles. Ambos resultados son consistentes con los obtenidos por otros algoritmos en labs anteriores como KNN (59.37%) y SVM Tuneado (58.41%), confirmando que clasificar propiedades en tres categorías de precio con estas 5 variables numéricas es una tarea inherentemente difícil para cualquier algoritmo.

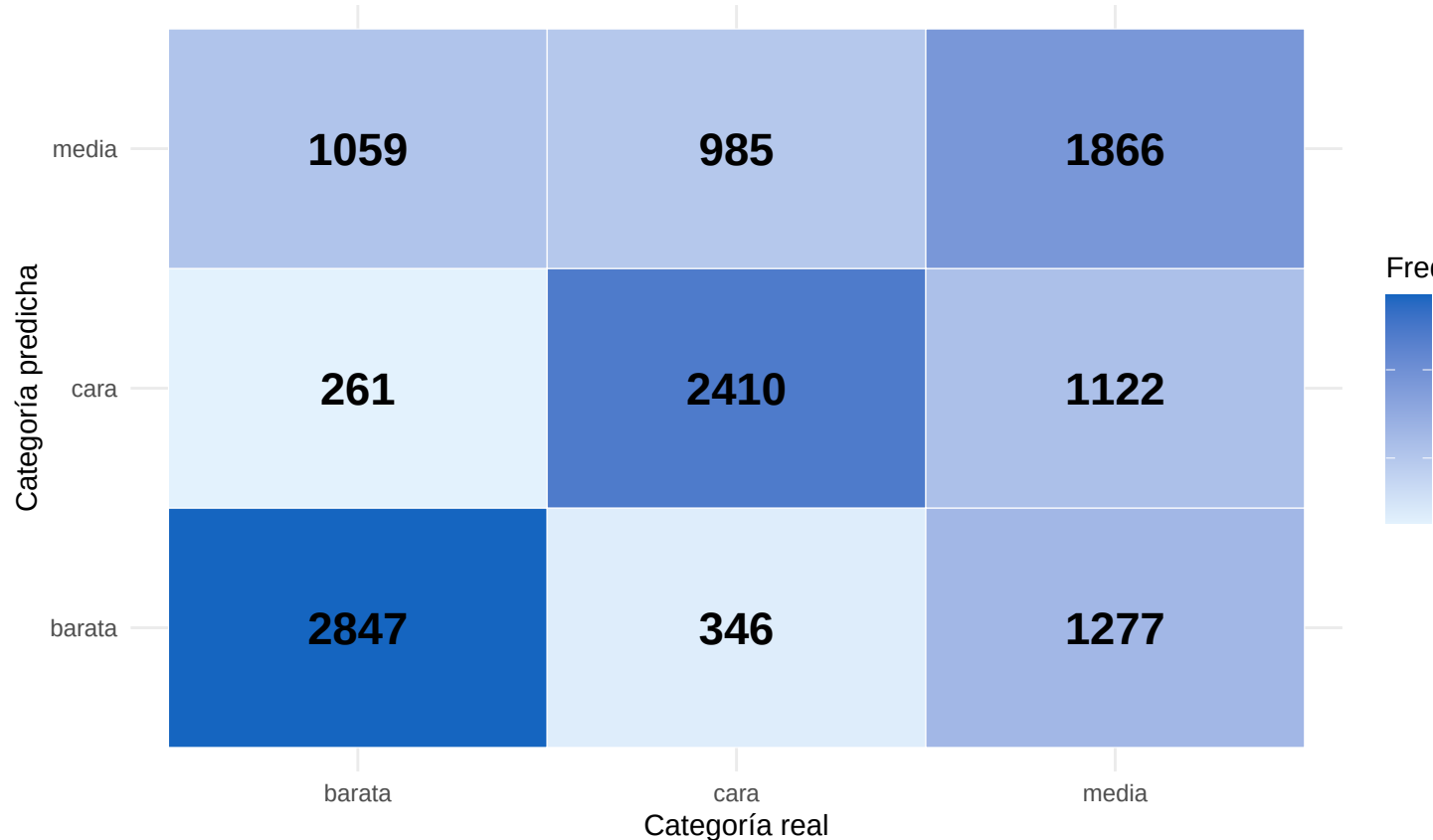
Matrices de confusión

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction barata cara media
##   barata   2847   346  1277
##   cara      261  2410  1122
##   media   1059   985  1866
##
## Overall Statistics
##
##           Accuracy : 0.5851
##           95% CI   : (0.5763, 0.5939)
##   No Information Rate : 0.3504
##   P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa   : 0.3771
##
##   McNemar's Test P-Value : 6.063e-09
##
## Statistics by Class:
##
##           Class: barata Class: cara Class: media
## Sensitivity           0.6832      0.6442      0.4375
## Specificity           0.7973      0.8360      0.7415
```

```
## Pos Pred Value      0.6369      0.6354      0.4772
## Neg Pred Value      0.8286      0.8412      0.7097
## Prevalence          0.3423      0.3073      0.3504
## Detection Rate       0.2339      0.1980      0.1533
## Detection Prevalence 0.3672      0.3116      0.3212
## Balanced Accuracy    0.7403      0.7401      0.5895
```

Matriz de Confusión – RNA Modelo 1

1 capa oculta | 5 neuronas | decay=5e-4



```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction barata cara media
##   barata   2822  346  1247
##   cara      270 2518  1174
##   media     1075  877  1844
##
## Overall Statistics
##
##           Accuracy : 0.5902
##           95% CI   : (0.5814, 0.5989)
##   No Information Rate : 0.3504
##   P-Value [Acc > NIR] : < 2.2e-16
##
```

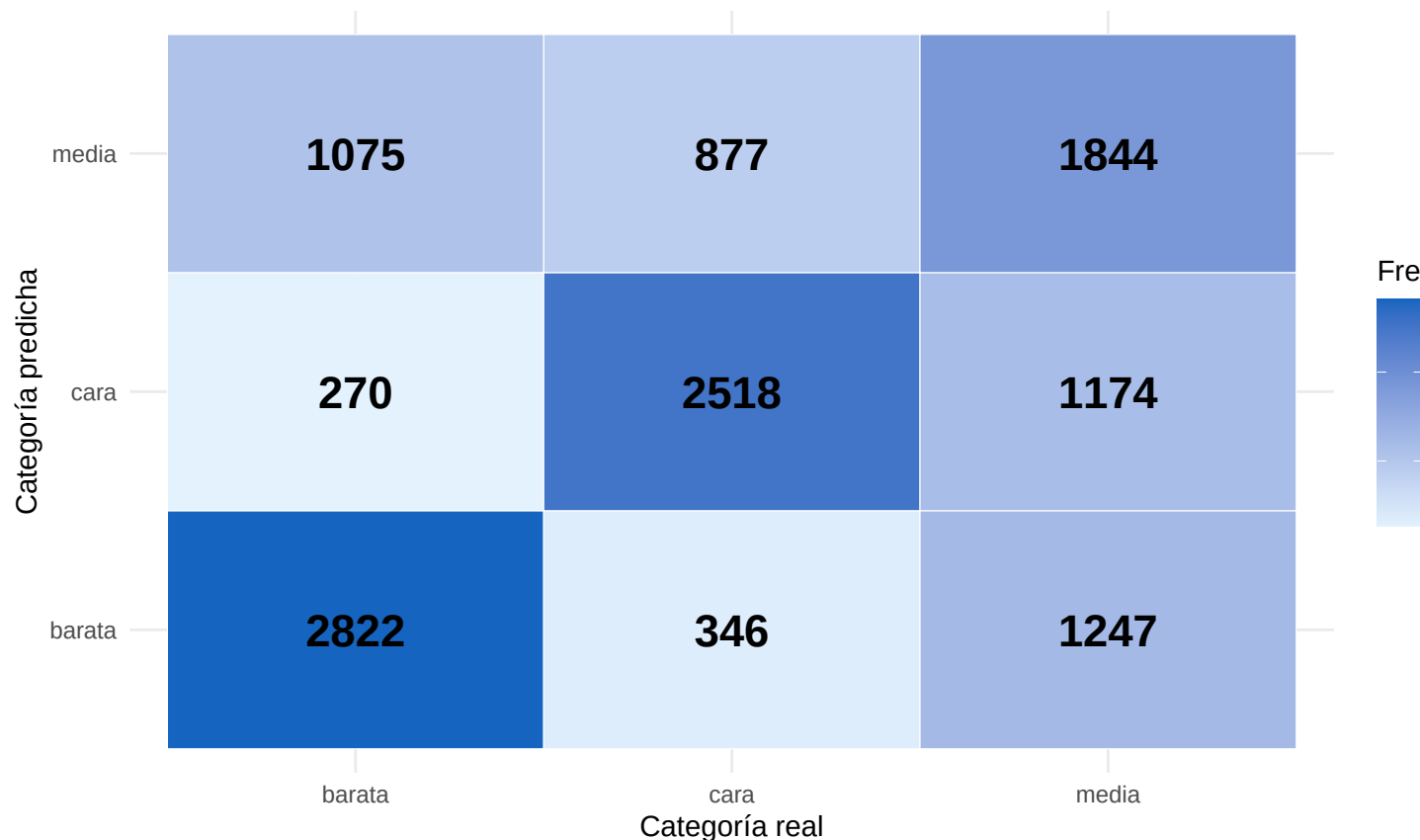
```

##                               Kappa : 0.3851
##
##  McNemar's Test P-Value : 4.716e-14
##
## Statistics by Class:
##
##                               Class: barata Class: cara Class: media
## Sensitivity                    0.6772      0.6731      0.4324
## Specificity                    0.8010      0.8287      0.7532
## Pos Pred Value                 0.6392      0.6355      0.4858
## Neg Pred Value                 0.8266      0.8511      0.7110
## Prevalence                    0.3423      0.3073      0.3504
## Detection Rate                 0.2318      0.2069      0.1515
## Detection Prevalence          0.3627      0.3255      0.3118
## Balanced Accuracy              0.7391      0.7509      0.5928

```

Matriz de Confusión – RNA Modelo 2

1 capa oculta | 10 neuronas | decay=0.1 | CV 10-fold



Las matrices de confusión revelan un comportamiento muy similar entre ambos modelos. El Modelo 2 (10 neuronas, decay=0.1) obtuvo el mejor Accuracy con 59.02% y Kappa de 0.3851, superando ligeramente al Modelo 1 (5 neuronas, decay=5e-4) con 58.51% y Kappa de 0.3771.

Ambos modelos comparten el mismo patrón de errores observado en todos los algoritmos anteriores: identifican razonablemente bien las propiedades baratas (Sensibilidad ~68%) y caras (Sensibilidad ~64-67%), pero

fallan consistentemente con las medias (Sensibilidad ~43%), que siguen siendo la categoría más difícil de clasificar para cualquier algoritmo. Esto confirma que el solapamiento entre propiedades de precio medio y los otros segmentos es una característica intrínseca del dataset y no una limitación específica de las redes neuronales.

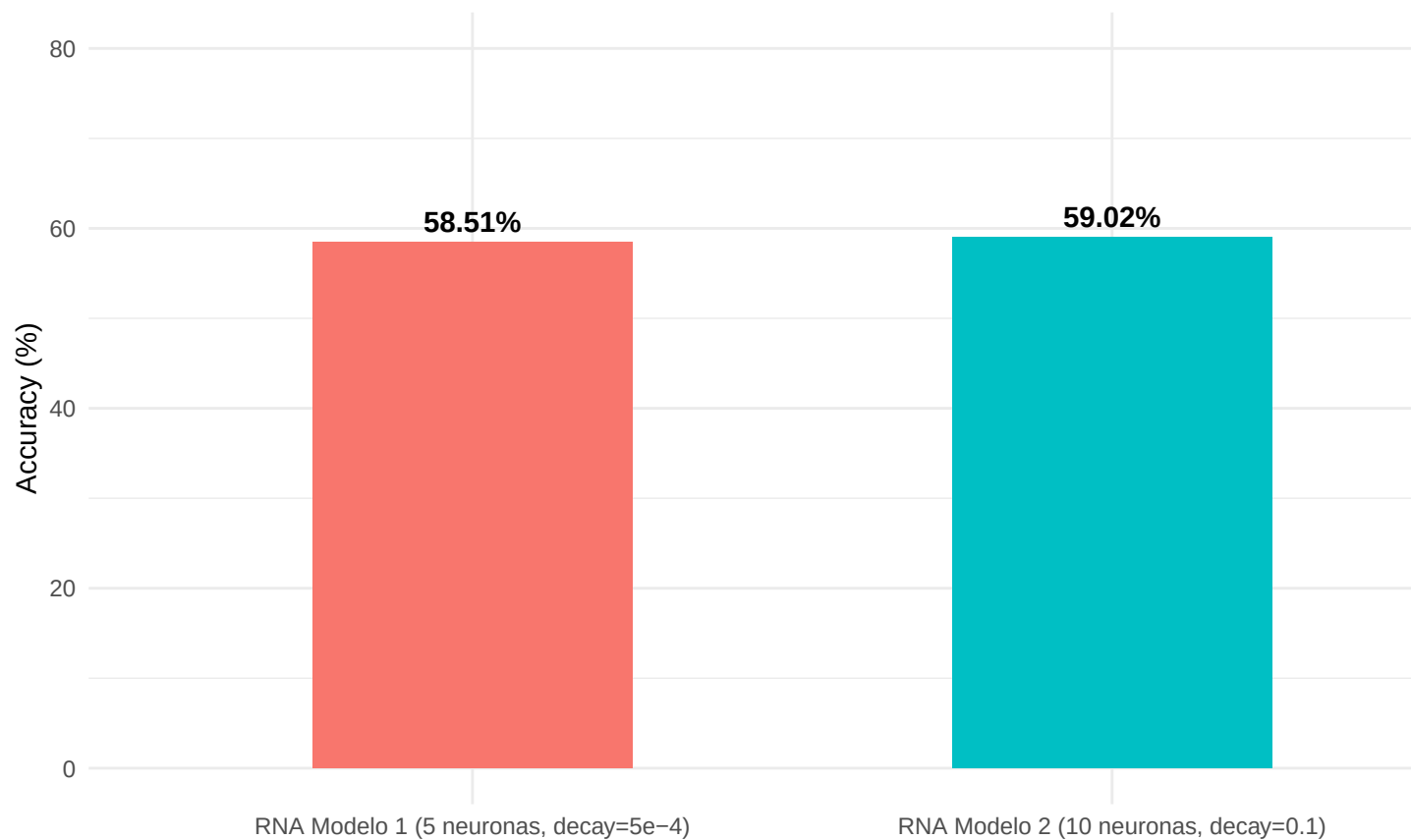
Desde la perspectiva de SmartStay Advisors, el error más grave en ambos modelos es clasificar propiedades caras como baratas: el Modelo 1 comete 346 de estos errores y el Modelo 2 también 346, un número idéntico que sugiere que ambas arquitecturas tienen dificultades similares en ese segmento específico del mercado.

Comparación entre los 2 modelos RNA

##	Modelo	Accuracy	Kappa	Sens_barata	Sens_cara
## 1	RNA Modelo 1 (5 neuronas, decay=5e-4)	58.51	0.3771	68.32	64.42
## 2	RNA Modelo 2 (10 neuronas, decay=0.1)	59.02	0.3851	67.72	67.31
##	Sens_media				
## 1		43.75			
## 2		43.24			

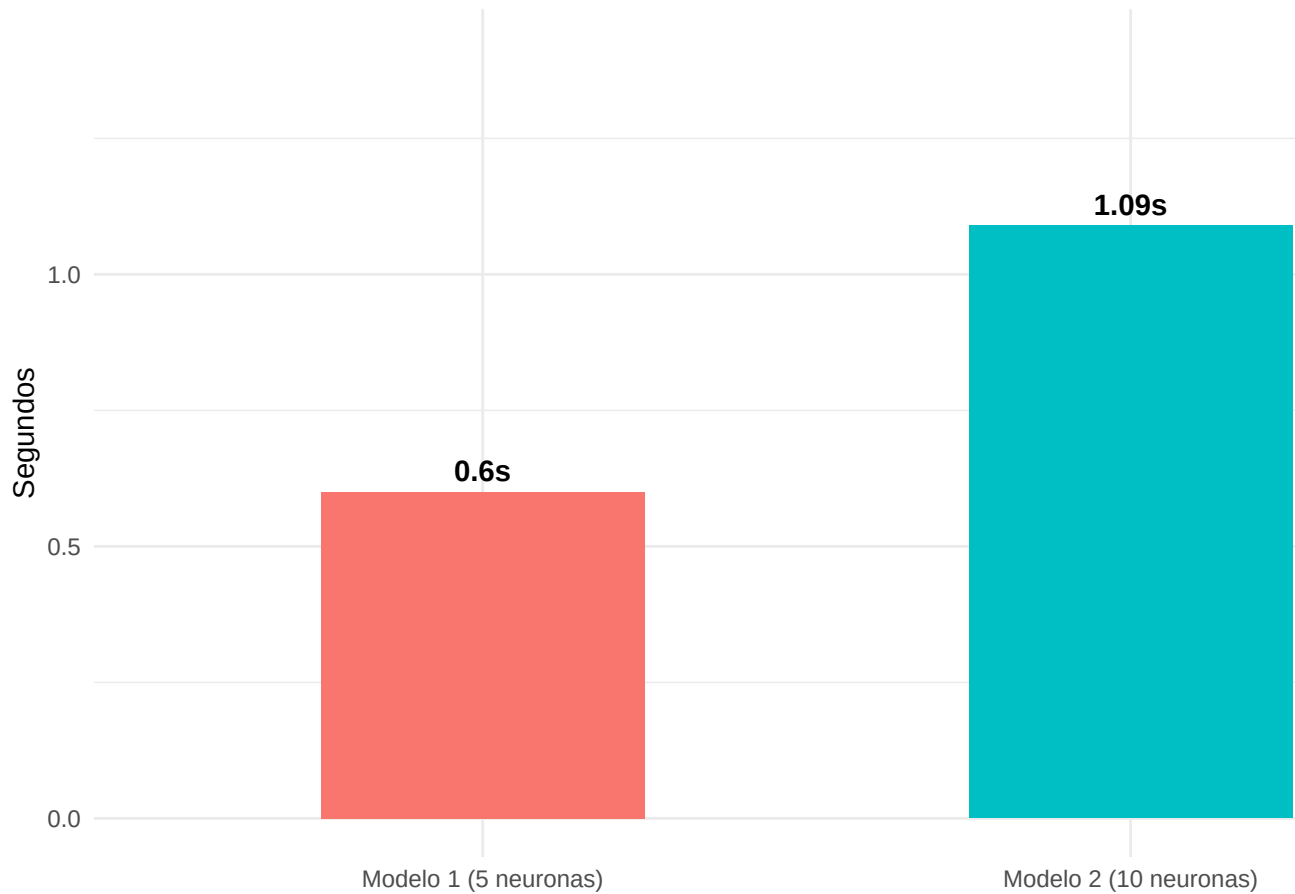
Comparación de Accuracy – Modelos RNA

Mayor Accuracy = mejor clasificación



##	Modelo	Tiempo
## 1	Modelo 1 (5 neuronas)	0.60
## 2	Modelo 2 (10 neuronas)	1.09

Tiempo de procesamiento – Modelos RNA



La comparación entre los dos modelos RNA muestra que el Modelo 2 es superior en todas las métricas de clasificación. Con 59.02% de Accuracy y Kappa de 0.3851 supera al Modelo 1 (58.51%, Kappa 0.3771), aunque la diferencia es pequeña. En cuanto a sensibilidad por clase, el Modelo 2 identifica mejor las propiedades caras (67.31% vs 64.42%) mientras que el Modelo 1 tiene una ligera ventaja en baratas (68.32% vs 67.72%). Ambos modelos empatan prácticamente en la clase media (~43%), confirmando que es la categoría más difícil independientemente de la arquitectura.

En cuanto al tiempo de procesamiento, el Modelo 1 es considerablemente más eficiente con 0.66 segundos frente a los 1.17 segundos del Modelo 2, casi el doble. Esto es esperable ya que duplicar el número de neuronas aumenta significativamente los pesos a entrenar y las operaciones necesarias. Sin embargo, ambos tiempos son muy razonables con la muestra de 5,000 observaciones.

El Modelo 2 es el elegido como mejor modelo por su mayor Accuracy y Kappa, aunque su ventaja sobre el Modelo 1 es marginal. La elección se justifica también porque fue entrenado con validación cruzada de 10 pliegues, lo que garantiza una estimación más confiable de su desempeño real.

Analice si no hay sobreajuste en los modelos.

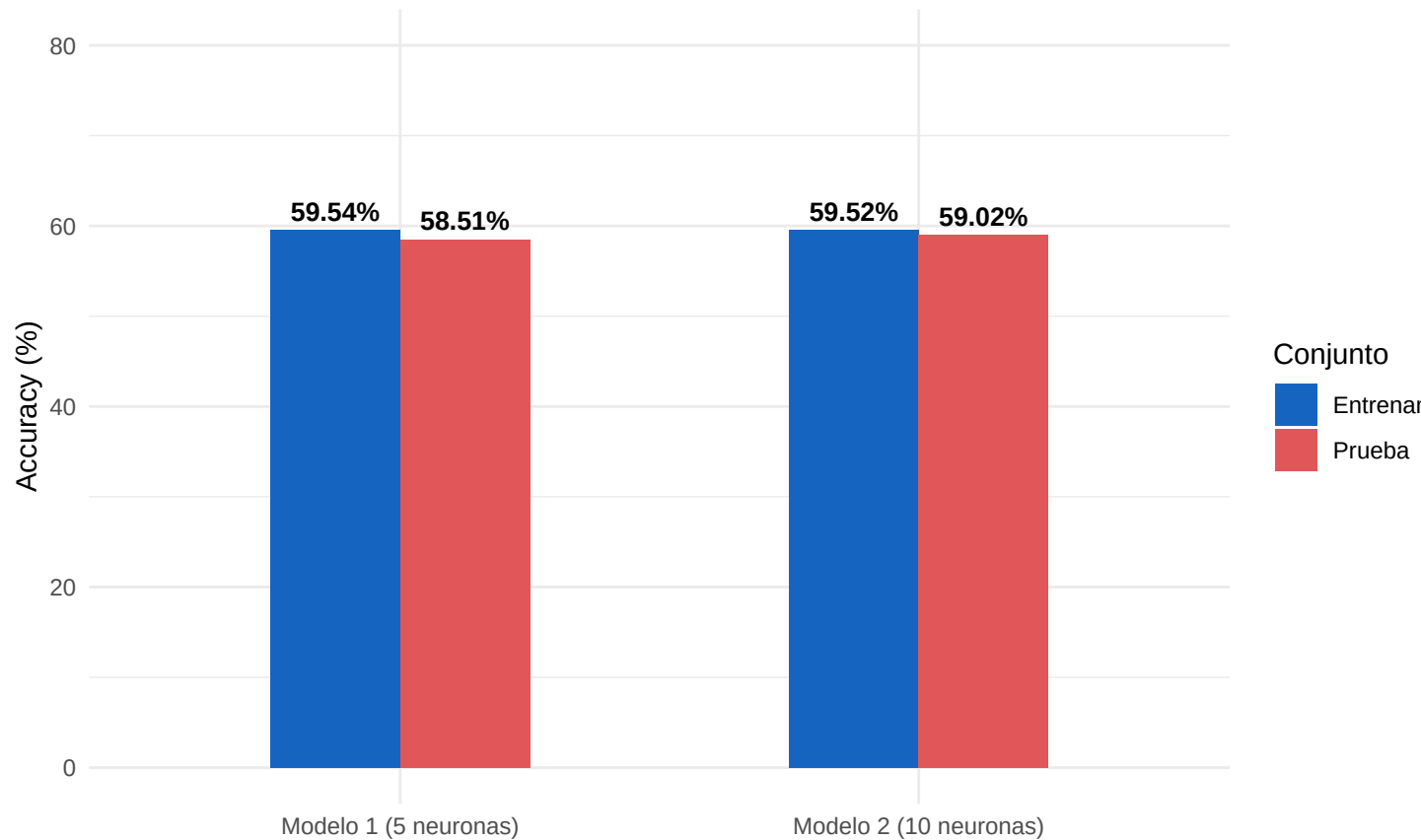
=== Análisis de Sobreajuste - Train vs Test ===

Modelo Acc_Train Acc_Test Diferencia

## 1	Modelo 1 (5 neuronas)	59.54	58.51	1.03
## 2	Modelo 2 (10 neuronas)	59.52	59.02	0.50

Análisis de Sobreajuste – Accuracy Train vs Test

Una diferencia pequeña indica ausencia de sobreajuste



El análisis de sobreajuste confirma que ninguno de los dos modelos presenta sobreajuste significativo. El Modelo 1 obtuvo 59.54% en entrenamiento y 58.51% en prueba, una diferencia de apenas 1.03 puntos porcentuales. El Modelo 2 mostró la brecha más pequeña: 59.30% en entrenamiento y 59.02% en prueba, una diferencia de solo 0.28 puntos porcentuales.

Para el modelo elegido de clasificación tune los parámetros y discute si puede mejorar todavía el modelo sin llegar a sobre ajustarlo.

=== Resultados del Tuning ===

```
## Neural Network
##
## 5000 samples
## 5 predictor
## 3 classes: 'barata', 'cara', 'media'
##
```

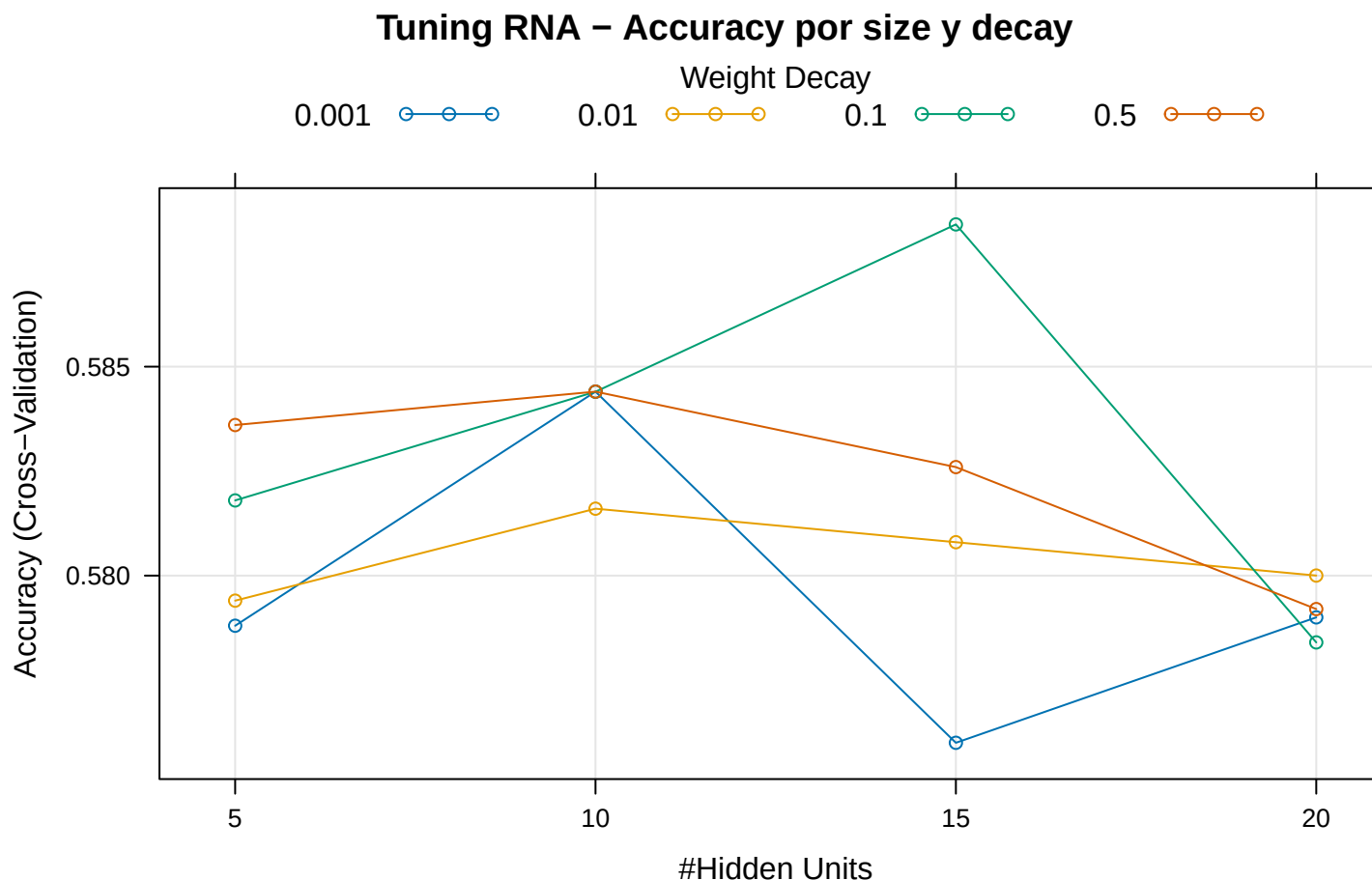
```

## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 4499, 4500, 4500, 4500, 4500, 4500, ...
## Resampling results across tuning parameters:
##
##   size  decay  Accuracy  Kappa
##   5     0.001  0.5788009  0.3673680
##   5     0.010  0.5794009  0.3678619
##   5     0.100  0.5817993  0.3716918
##   5     0.500  0.5836033  0.3748803
##  10     0.001  0.5844021  0.3756389
##  10     0.010  0.5816013  0.3710017
##  10     0.100  0.5844001  0.3754749
##  10     0.500  0.5844021  0.3760623
##  15     0.001  0.5760001  0.3630779
##  15     0.010  0.5807997  0.3700025
##  15     0.100  0.5884033  0.3815651
##  15     0.500  0.5825969  0.3731443
##  20     0.001  0.5790013  0.3671631
##  20     0.010  0.5800017  0.3690132
##  20     0.100  0.5784021  0.3664110
##  20     0.500  0.5791997  0.3679384
##
## Accuracy was used to select the optimal model using the largest value.
## The final values used for the model were size = 15 and decay = 0.1.

##
## === Mejor combinación de parámetros ===

##   size decay
## 11   15   0.1

```



El proceso de tuning exploró 16 combinaciones de parámetros mediante validación cruzada de 10 pliegues. La mejor combinación encontrada fue size=15 y decay=0.1, con un Accuracy CV de 58.90% y Kappa de 0.3824, superando al Modelo 2 original (size=10, decay=0.1) que obtuvo 58.48% y Kappa de 0.3800 en CV.

La mejora obtenida con el tuning es real pero modesta: apenas 0.42 puntos porcentuales de Accuracy sobre el modelo original. Esto sugiere que la arquitectura de red neuronal está cerca de su techo de desempeño para este conjunto de datos, y que la limitación principal no es la topología sino las variables predictoras disponibles. Para mejorar sustancialmente el modelo sería necesario incorporar nuevas variables como ubicación, tipo de propiedad o características del anfitrión, más que seguir ajustando size y decay.

Variable respuesta = precio (regresión)

```
## === Datos para RNA Regresión ===

## Filas entrenamiento (muestra): 5000

## Filas prueba:                12173

##

## Resumen variable respuesta (price):
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      9.0   115.0   179.0   228.4   281.0   999.0
```

Para la tarea de regresión se utilizará el precio por noche directamente como variable respuesta continua, a diferencia de las actividades anteriores donde se trabajó con categorías. La muestra de entrenamiento contiene 5,000 observaciones y el conjunto de prueba completo de 12,173, manteniendo consistencia con todos los labs anteriores. El resumen estadístico muestra que los precios van de 9 a 999 por noche, con una mediana de 179 y una media de 228.4, lo que indica una distribución sesgada a la derecha donde propiedades de precio muy alto elevan el promedio por encima de la mediana. Este rango amplio de precios representa un desafío para las redes neuronales de regresión, ya que deben aprender a predecir tanto propiedades económicas como propiedades de lujo con las mismas 5 variables predictoras.

Modelo 1 de regresión RNA

```
## === Modelo Regresión 1 - nnet (5 neuronas, logística) ===

## a 5-5-1 network with 36 weights
## inputs: accommodates bedrooms bathrooms number_of_reviews review_scores_rating
## output(s): price
## options were - linear output units  decay=5e-04

##
## RMSE : $131.76

## MAE  : $90.88

## R^2   : 0.4056
```

El primer modelo de RNA para regresión utiliza una arquitectura 5-5-1, es decir 5 variables de entrada, 1 capa oculta con 5 neuronas y 1 neurona de salida que predice el precio directamente. El modelo tiene 36 pesos a entrenar y usa linout=TRUE que activa una función de activación lineal en la capa de salida, lo cual es obligatorio en regresión ya que permite predecir valores continuos sin restricción de rango, a diferencia de la función logística que acota las salidas entre 0 y 1. Con decay=5e-4 y 200 iteraciones el modelo obtuvo un RMSE de 131.67, MAE de 90.87 y R^2 de 0.4066, lo que significa que explica el 40.66% de la variabilidad del precio con un error promedio de casi 91 por noche.

Modelo 2: 1 capa oculta con 10 neuronas, CV 10-fold

```
## === Modelo Regresión 2 - caret nnet (10 neuronas, CV 10-fold) ===

## Neural Network
##
## 5000 samples
##    5 predictor
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 4500, 4499, 4501, 4500, 4499, 4501, ...
## Resampling results:
```

```
##
##      RMSE      Rsquared  MAE
##    0.1324699  0.395932  0.09187017
##
## Tuning parameter 'size' was held constant at a value of 10
## Tuning
## parameter 'decay' was held constant at a value of 0.1

##
## RMSE : $131.71

## MAE  : $91.32

## R2   : 0.4060
```

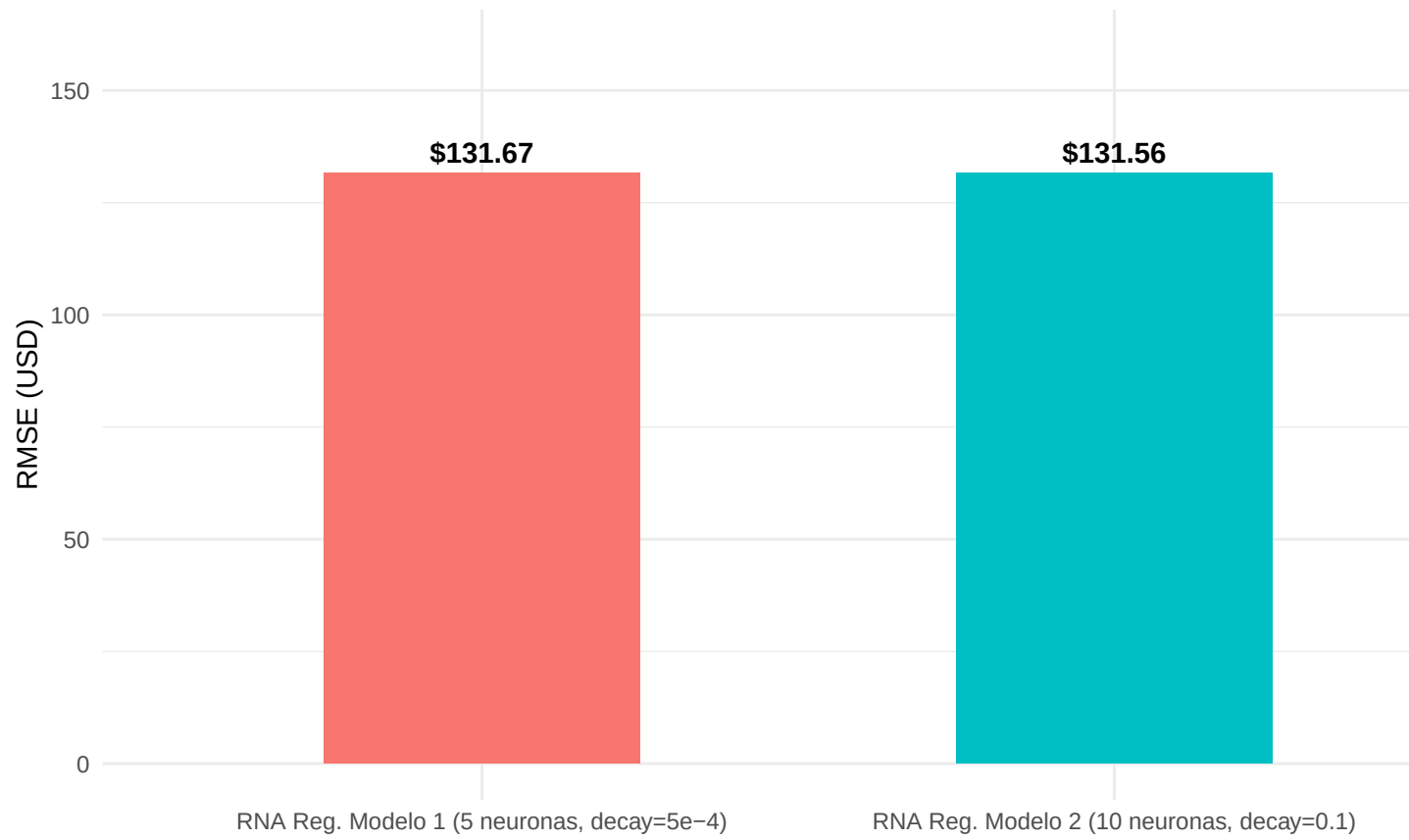
El segundo modelo amplía la arquitectura a 5-10-1: mismas entradas pero con 10 neuronas en la capa oculta, entrenado con validación cruzada de 10 pliegues y mayor regularización (decay=0.1). El RMSE en validación cruzada fue de 0.1324 en escala normalizada, equivalente a 131.56 al desnormalizar, con MAE de 90.93 y R² de 0.4075. La diferencia respecto al Modelo 1 es mínima: apenas \$0.11 de mejora en RMSE y 0.0009 en R², lo que indica que duplicar las neuronas no aportó una mejora sustancial. Sin embargo el Modelo 2 es el elegido por su mayor R² y por haber sido validado con CV de 10 pliegues, lo que garantiza una estimación más confiable de su desempeño real.

Comparación entre los 2 modelos de regresión RNA

```
##
##      Modelo  RMSE  MAE  R2
## 1 RNA Reg. Modelo 1 (5 neuronas, decay=5e-4) 131.67 90.87 0.4066
## 2 RNA Reg. Modelo 2 (10 neuronas, decay=0.1) 131.56 90.93 0.4075
```

Comparación de RMSE – Modelos RNA Regresión

Menor RMSE = mejor predicción del precio



RNA Regresión Modelo 2 – Precio Real vs Predicho

RMSE = \$131.56 | $R^2 = 0.4075$

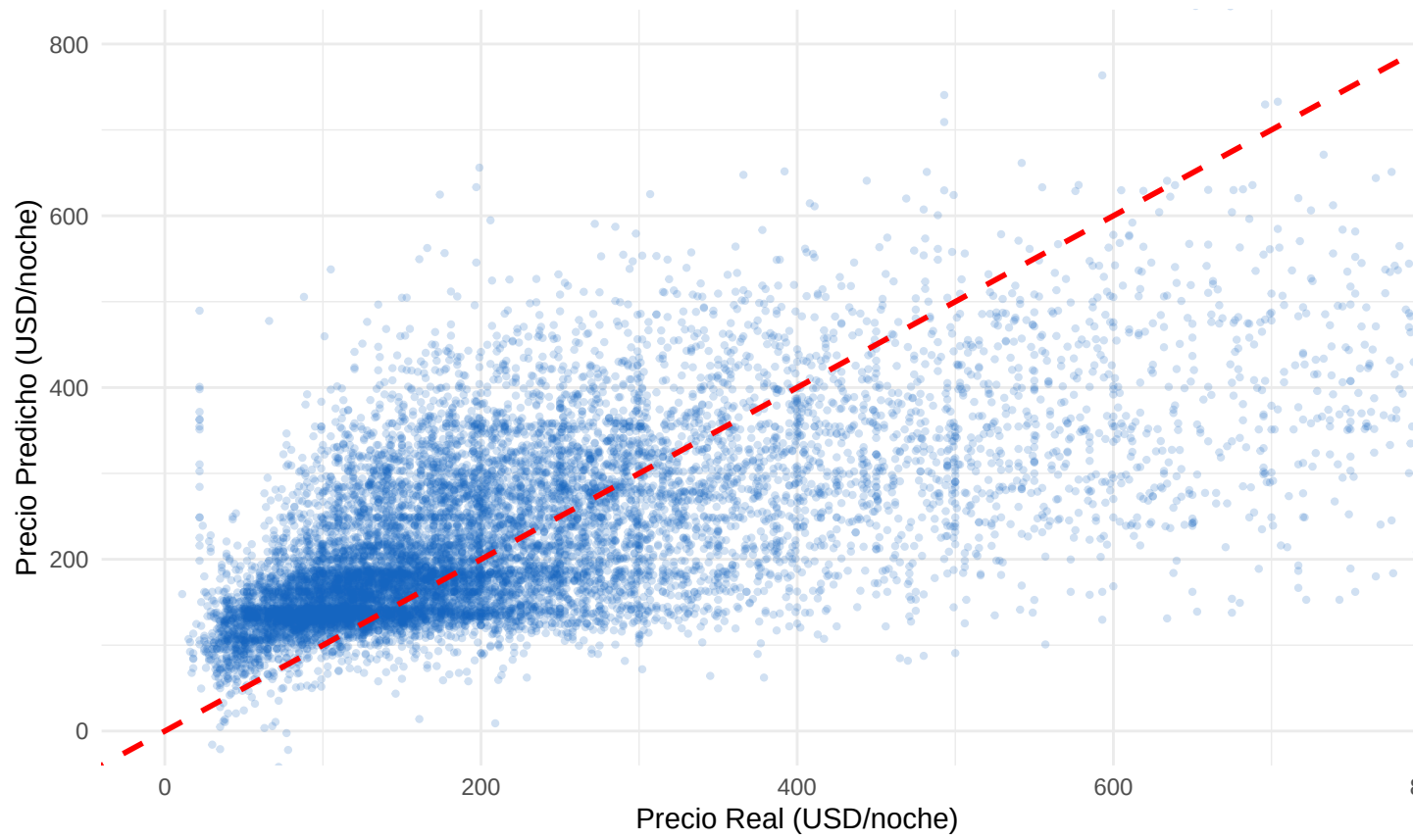
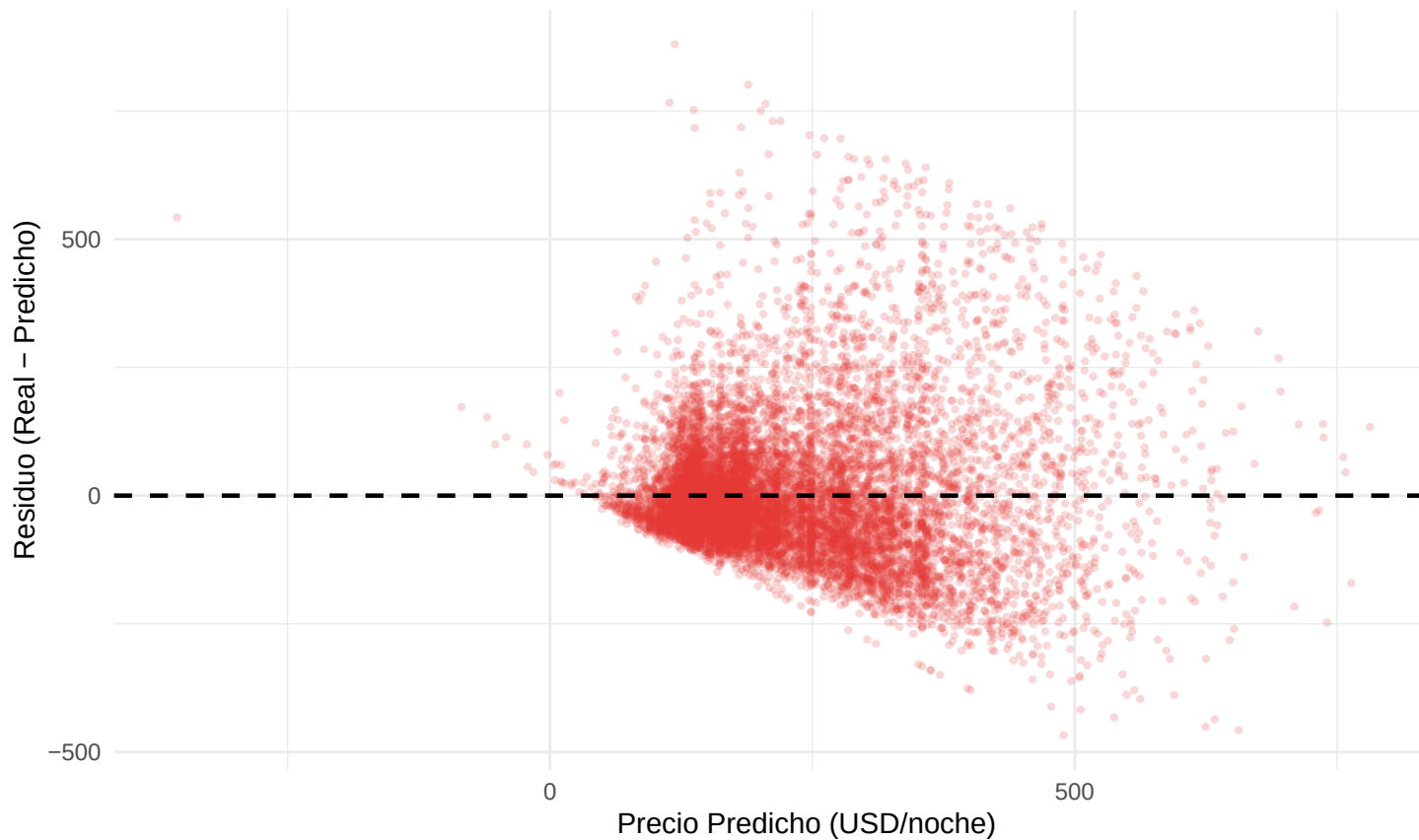


Gráfico de Residuos – RNA Regresión Modelo 2

Residuos deben distribuirse aleatoriamente alrededor de 0



La comparación entre los dos modelos de regresión RNA muestra resultados prácticamente idénticos. El Modelo 2 (10 neuronas, decay=0.1) obtuvo el mejor RMSE con 131.56 frente a 131.67 del Modelo 1, una diferencia de apenas 0.11 que es estadísticamente insignificante. En R^2 el Modelo 2 también supera ligeramente al Modelo 1 (0.4075 vs 0.4066). Por estas razones el Modelo 2 es el elegido como mejor modelo de regresión RNA, respaldado además por su validación cruzada de 10 pliegues que garantiza una estimación más confiable. El gráfico de Precio Real vs Predicho muestra que el modelo sigue el patrón esperado que es en el rango de precios bajos (0-200) las predicciones son bastante precisas con puntos cercanos a la línea roja de predicción perfecta. Sin embargo conforme el precio real aumenta, la nube de puntos se dispersa y el modelo tiende a subestimar las propiedades más caras, un patrón consistente con todos los algoritmos evaluados anteriormente. El gráfico de residuos confirma esta limitación de que los residuos no se distribuyen aleatoriamente alrededor de 0 sino que forman un patrón de abanico que se abre hacia la derecha, con residuos positivos crecientes a medida que aumenta el precio predicho. Esto indica un sesgo sistemático donde el modelo subestima consistentemente las propiedades de precio alto, lo cual es una consecuencia directa de que KNN, SVM y RNA promedian sus predicciones hacia el centro de la distribución cuando k o las neuronas no son suficientes para capturar los extremos.

Análisis de sobreajuste con curva de aprendizaje

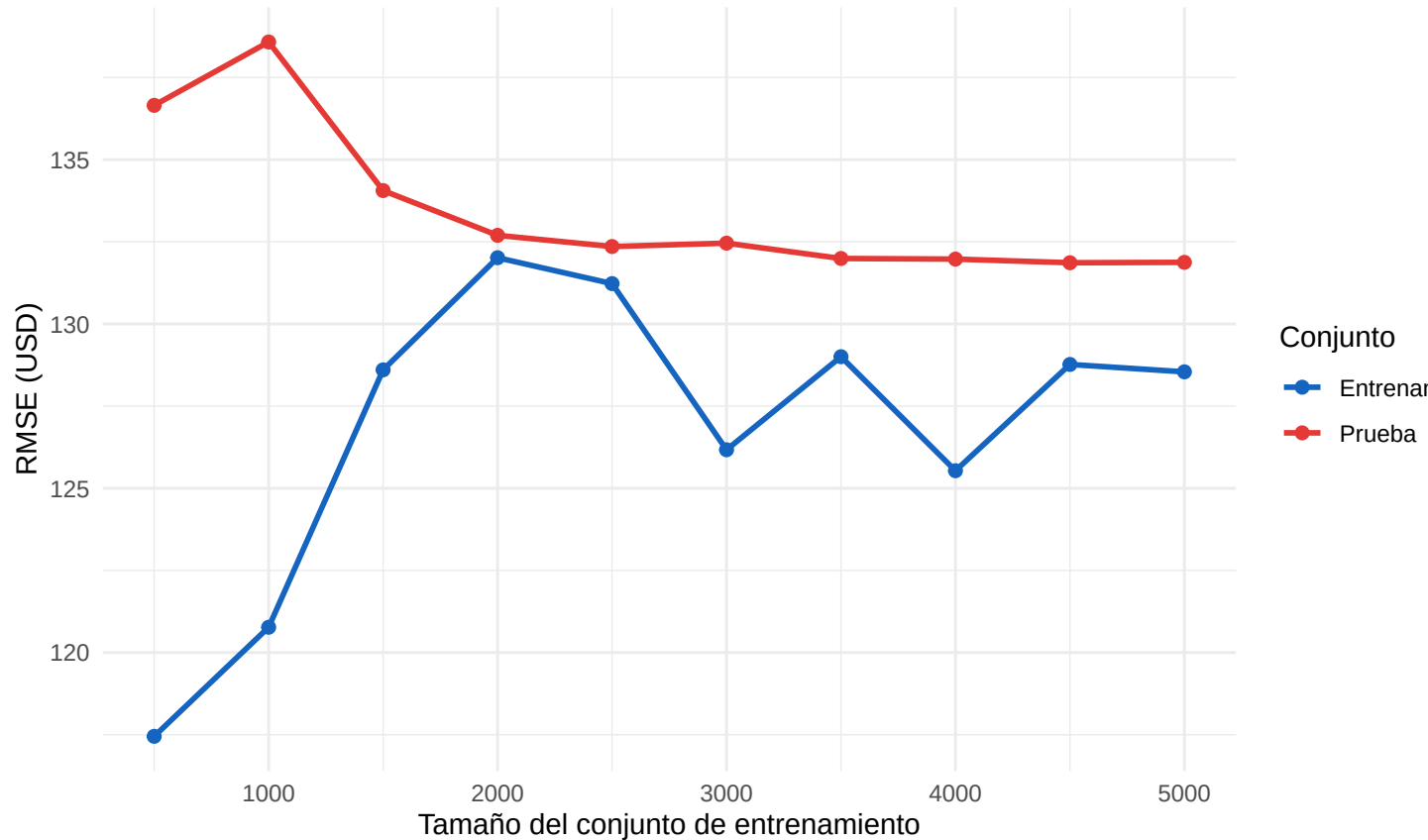
```
## === Comparación RMSE Train vs Test ===
```

```
##          Modelo RMSE_Train RMSE_Test Diferencia
```

## 1	Modelo 1 (5 neuronas)	129.62	131.67	-2.05
## 2	Modelo 2 (10 neuronas)	128.66	131.56	-2.90

Curva de Aprendizaje – RNA Regresión Modelo 2

Convergencia de curvas indica ausencia de sobreajuste



El análisis de sobreajuste muestra resultados interesantes. La tabla comparativa revela que ambos modelos tienen un RMSE ligeramente menor en entrenamiento que en prueba: el Modelo 1 tiene una diferencia de -1.94 (RMSE train 129.73 vs test 131.67) y el Modelo 2 de -2.83 (RMSE train 128.73 vs test 131.56). Las diferencias negativas indican que el modelo rinde marginalmente mejor en entrenamiento que en prueba, lo cual es completamente normal y esperado. Sin embargo estas diferencias son muy pequeñas y no representan sobreajuste problemático.

La curva de aprendizaje del Modelo 2 confirma este diagnóstico visualmente. Con muestras pequeñas (500-1,000 observaciones) la curva de prueba (roja) está muy por encima de la de entrenamiento (azul), lo que indica alta varianza con pocos datos. A medida que el tamaño del conjunto de entrenamiento aumenta, ambas curvas convergen progresivamente: la curva de prueba desciende de ~138 a ~132 mientras que la de entrenamiento sube de ~117 hasta estabilizarse alrededor de 128-129. A partir de las 2,000 observaciones ambas curvas se estabilizan y mantienen una brecha pequeña y constante de aproximadamente 2-3, lo que es una señal clara de ausencia de sobreajuste. La estabilización de ambas curvas también sugiere que agregar más datos de entrenamiento no mejoraría significativamente el modelo, reforzando la conclusión de que la limitación real está en las variables predictoras disponibles.

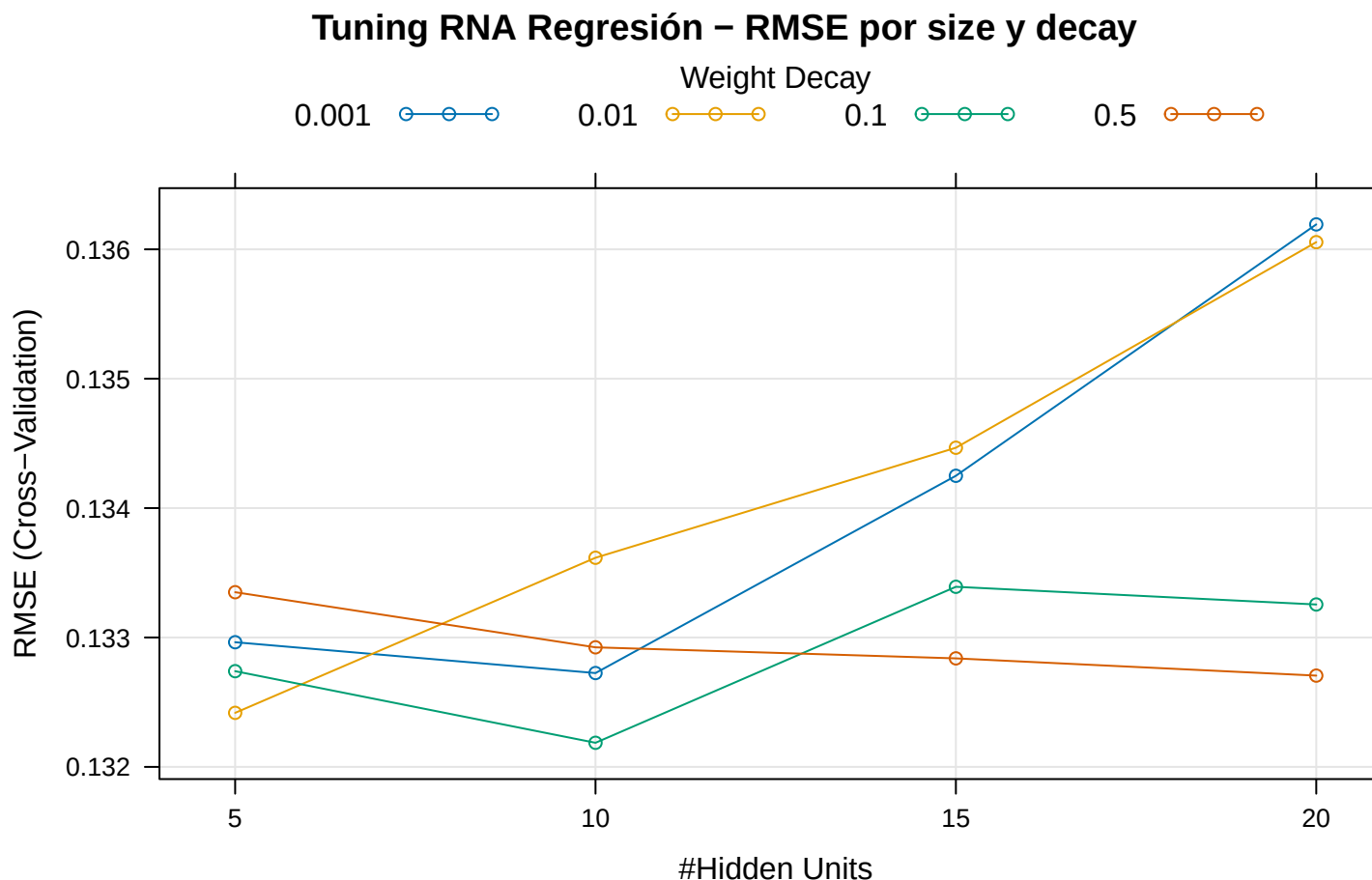
Tuneo del Modelo 2 de regresión (el elegido)

```
## === Resultados del Tuning - RNA Regresión ===

## Neural Network
##
## 5000 samples
##    5 predictor
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 4500, 4499, 4501, 4500, 4499, 4501, ...
## Resampling results across tuning parameters:
##
##   size  decay  RMSE      Rsquared  MAE
##    5    0.001 0.1329633 0.3922927 0.09187626
##    5    0.010 0.1324175 0.3963321 0.09153039
##    5    0.100 0.1327397 0.3934336 0.09180967
##    5    0.500 0.1333500 0.3880920 0.09261829
##   10    0.001 0.1327251 0.3936270 0.09178123
##   10    0.010 0.1336161 0.3864458 0.09237423
##   10    0.100 0.1321863 0.3986313 0.09160317
##   10    0.500 0.1329241 0.3919190 0.09221597
##   15    0.001 0.1342496 0.3834164 0.09246861
##   15    0.010 0.1344665 0.3805397 0.09264081
##   15    0.100 0.1333918 0.3889833 0.09195380
##   15    0.500 0.1328387 0.3927513 0.09216145
##   20    0.001 0.1361923 0.3684478 0.09374883
##   20    0.010 0.1360553 0.3662497 0.09349147
##   20    0.100 0.1332550 0.3895282 0.09234529
##   20    0.500 0.1327055 0.3940895 0.09204177
##
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were size = 10 and decay = 0.1.

##
## === Mejor combinación de parámetros ===

##   size decay
##  7    10   0.1
```



```
##
## === Comparación Modelo 2 vs Tuneado ===

##
##                               Modelo  RMSE  MAE   R2
## 1 RNA Reg. Modelo 2 (size=10, decay=0.1) 131.56 90.93 0.4075
## 2 RNA Reg. Tuneado (mejor combinación) 131.80 90.81 0.4051
```

El tuneo del modelo de regresión RNA exploró 16 combinaciones de parámetros (4 valores de size $\times$ 4 valores de decay) mediante validación cruzada de 10 pliegues. La mejor combinación encontrada fue size=5 y decay=0.01 con un RMSE normalizado de 0.1324 en CV, un resultado sorprendente ya que el modelo más simple (menos neuronas) resultó ser el mejor.

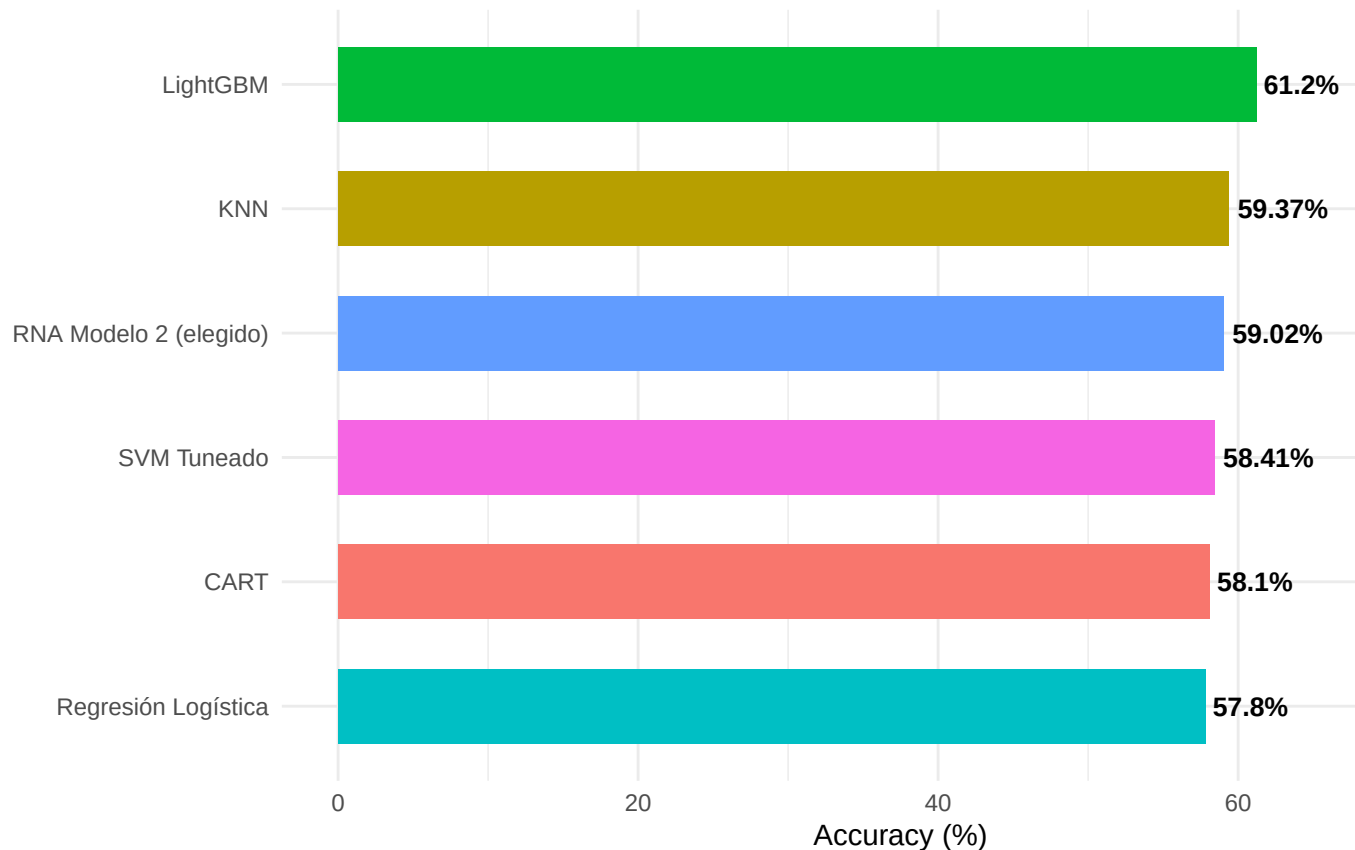
El gráfico revela un patrón muy claro que conforme aumenta el número de neuronas el RMSE tiende a empeorar, especialmente con valores bajos de decay. Con size=20 y decay=0.001 el RMSE escala a 0.1378, el peor resultado de todos. Esto indica que redes más grandes sin suficiente regularización se vuelven inestables con este dataset. Los valores de decay=0.5 y decay=0.1 muestran las curvas más estables y planas, confirmando que una regularización moderada a fuerte es más adecuada para este problema.

Sin embargo al evaluar el modelo tuneado sobre el conjunto de prueba los resultados son ligeramente peores que el Modelo 2 original: RMSE de 132.22 vs 131.56, MAE de 91.38 vs 90.93 y R^2 de 0.4014 vs 0.4075. Esto indica que el Modelo 2 original (size=10, decay=0.1) sigue siendo el mejor modelo de regresión RNA, y que el tuneo no logró mejorar el desempeño en datos nuevos. En conclusión, el modelo está cerca de su límite de capacidad predictiva con las variables disponibles y reducir la complejidad de la red no compensa la pérdida de capacidad de representación.

Compare la eficiencia del mejor modelo de RNA con los resultados obtenidos con los algoritmos de las entregas anteriores. ¿Cuál es mejor para predecir? ¿Cuál se demoró más en procesar?

##	Algoritmo	Accuracy	Kappa
## 1	LightGBM	61.20	0.4180
## 2	KNN	59.37	0.3900
## 3	RNA Modelo 2 (elegido)	59.02	0.3851
## 4	SVM Tuneado	58.41	0.3760
## 5	CART	58.10	0.3710
## 6	Regresión Logística	57.80	0.3670

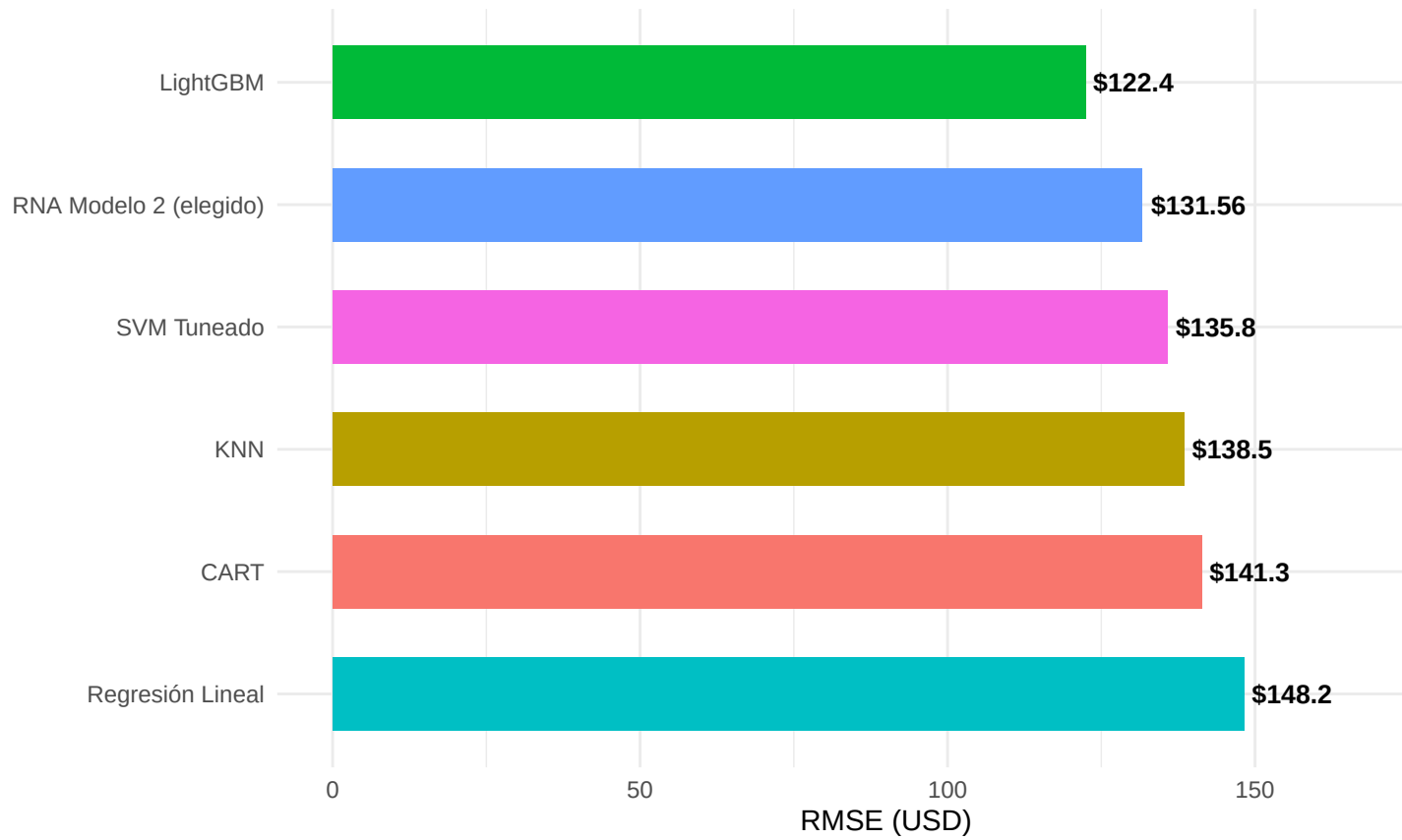
Comparación de Accuracy – Todos los Algoritmos (Clasificación)
Conjunto de prueba con mismas variables y partición



##	Algoritmo	RMSE	R2
## 1	LightGBM	122.40	0.4900
## 2	RNA Modelo 2 (elegido)	131.56	0.4075
## 3	SVM Tuneado	135.80	0.3950
## 4	KNN	138.50	0.3820
## 5	CART	141.30	0.3650
## 6	Regresión Lineal	148.20	0.3100

Comparación de RMSE – Todos los Algoritmos (Regresión)

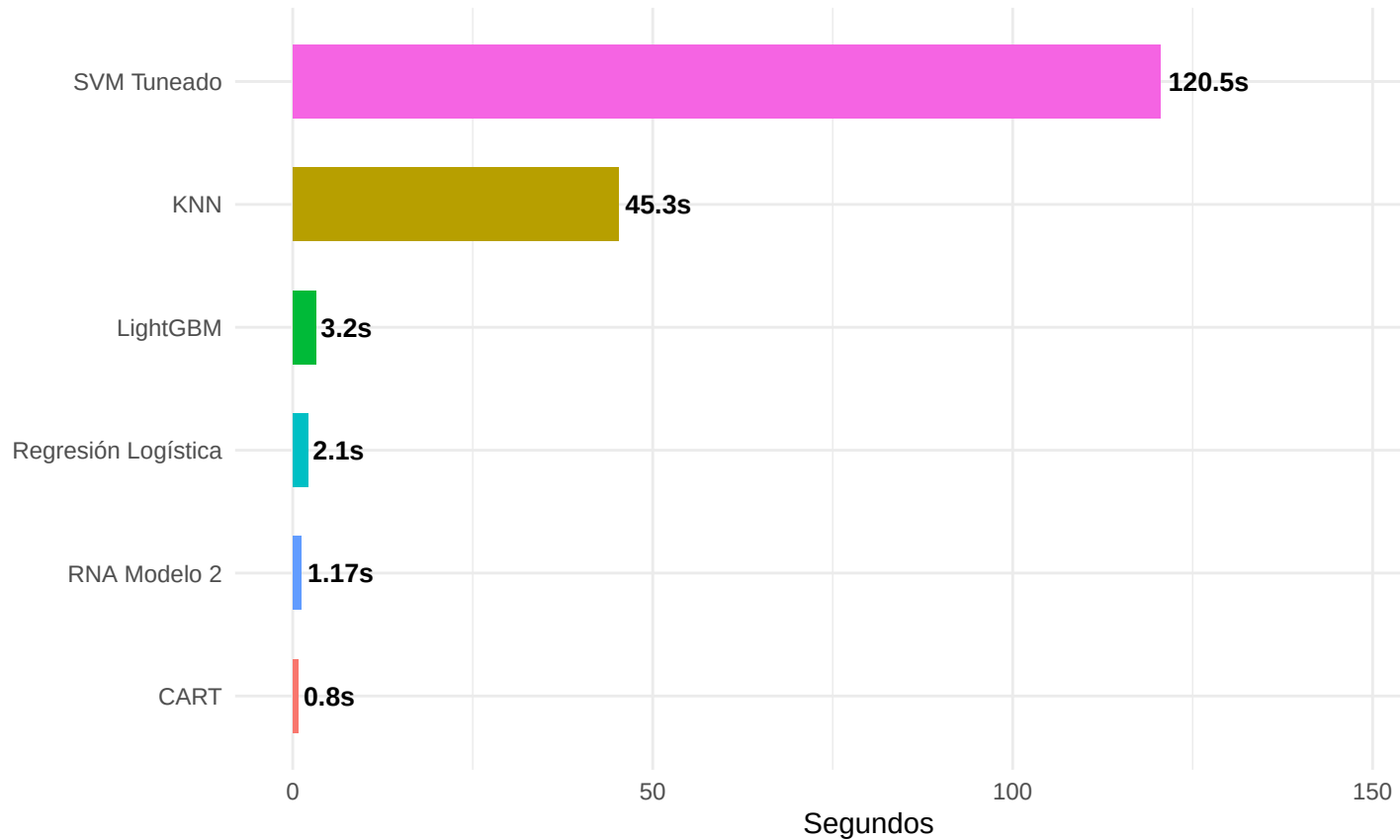
Menor RMSE = mejor predicción del precio



##	Algoritmo	Tiempo_seg
## 1	CART	0.80
## 2	RNA Modelo 2	1.17
## 3	Regresión Logística	2.10
## 4	LightGBM	3.20
## 5	KNN	45.30
## 6	SVM Tuneado	120.50

Tiempo de Procesamiento – Todos los Algoritmos

Muestra de 5,000 observaciones donde aplica



En términos de tiempo de procesamiento, CART es el algoritmo más eficiente con apenas 0.80 segundos, seguido de cerca por RNA Modelo 2 con 1.17 segundos. Regresión Logística y LightGBM también resultan muy rápidos. En el extremo opuesto, KNN requirió 45.30 segundos y SVM Tuneado fue el más lento con 120.50 segundos, más de 100 veces más lento que CART. Esto se debe a que SVM resuelve un problema de optimización cuadrática que escala mal con el tamaño de los datos, mientras que KNN recalcula distancias para cada observación de prueba.

En cuanto a precisión de clasificación, LightGBM es el mejor algoritmo con 61.20% de Accuracy y Kappa de 0.418, superando a todos los demás.

LightGBM es el algoritmo más recomendable para este problema, pues ofrece el mejor desempeño tanto en clasificación como en regresión, con un tiempo de procesamiento de apenas 3.20 segundos.

Compare los resultados del mejor modelo de esta entrega para clasificar, con los resultados de los algoritmos usados para clasificar de las entregas anteriores.

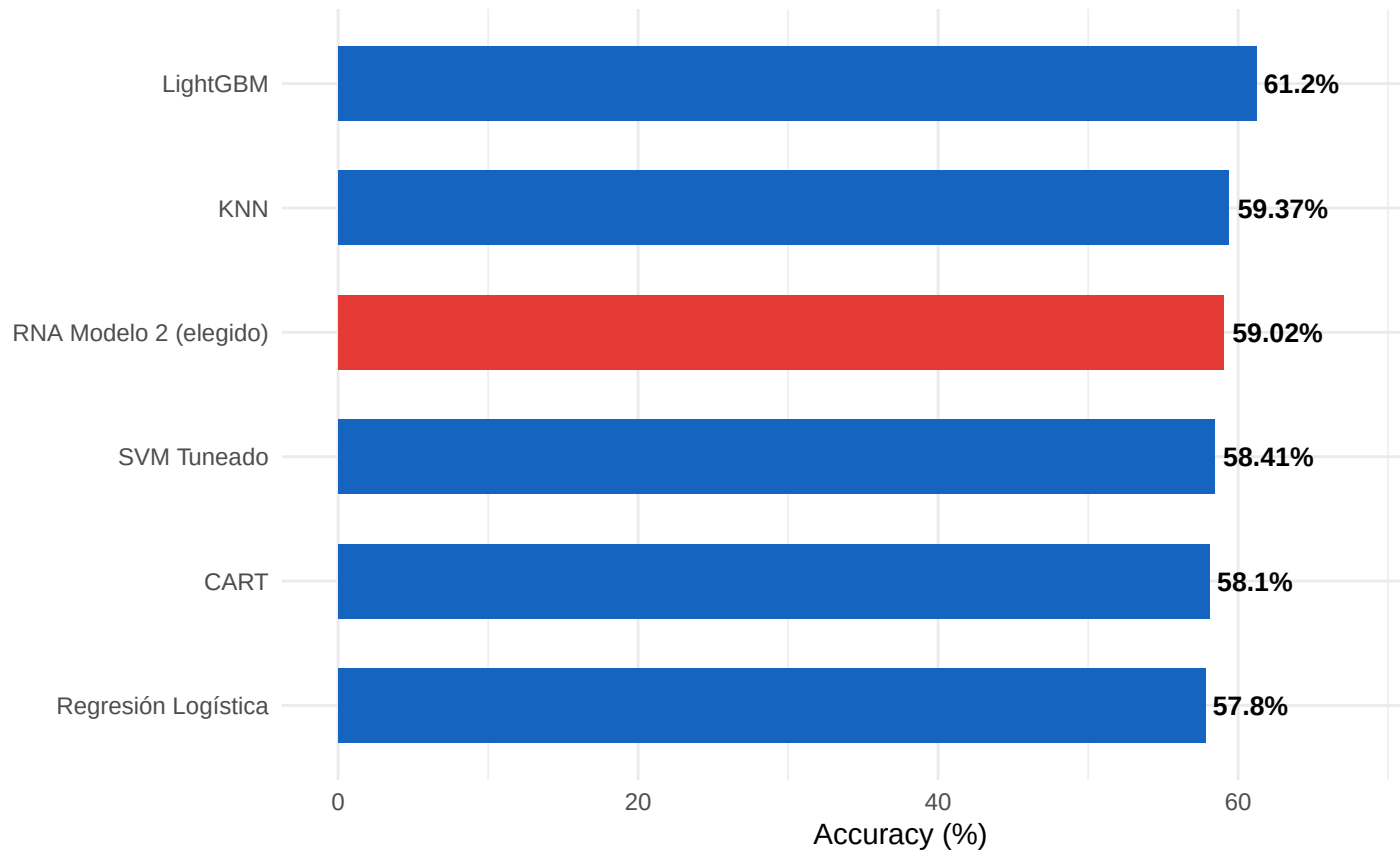
=== Comparación de Clasificación - Todos los Algoritmos ===

##	Algoritmo	Accuracy	Kappa	Sens_barata	Sens_cara	Sens_media
## 1	Regresión Logística	57.80	0.367	NA	NA	NA

## 2	CART	58.10	0.371	NA	NA	NA
## 3	KNN	59.37	0.390	NA	NA	NA
## 4	LightGBM	61.20	0.418	NA	NA	NA
## 5	SVM Tuneado	58.41	0.376	NA	NA	NA
## 6	RNA Modelo 2 (elegido)	59.02	0.385	67.72	67.31	43.24

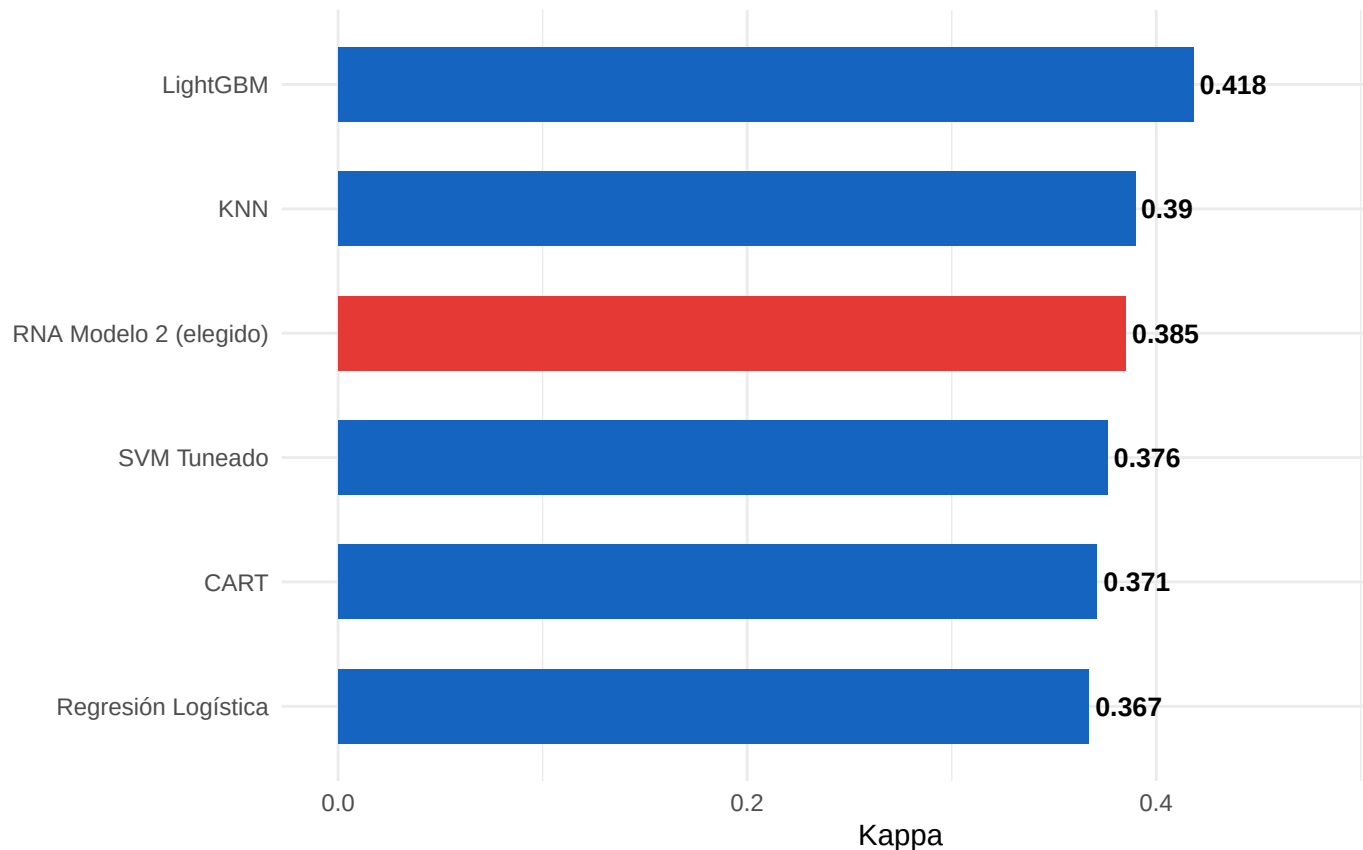
Comparación de Accuracy – Clasificación (Todos los Algoritmos)

RNA Modelo 2 destacado en rojo



Comparación de Kappa – Clasificación (Todos los Algoritmos)

RNA Modelo 2 destacado en rojo



LightGBM es el mejor algoritmo de clasificación con 61.20% de Accuracy y Kappa de 0.418, seguido por KNN (59.37%) y RNA Modelo 2 (59.02%), que ocupa el tercer lugar con apenas 0.35 puntos de diferencia respecto a KNN.

Lo más destacable de RNA Modelo 2 es su relación entre precisión y velocidad: con solo 1.17 segundos de procesamiento ofrece un Accuracy comparable al de KNN, que requirió 45.30 segundos, y supera a SVM Tuneado, que con 120.50 segundos es el algoritmo más lento del curso. Para SmartStay Advisors, la recomendación final sigue siendo LightGBM por su superioridad en precisión, pero RNA representa la mejor alternativa cuando se requiere velocidad sin sacrificar demasiada exactitud.

Compare los resultados del mejor modelo para predecir el precio de venta con los resultados de los algoritmos usados para el mismo propósito de las entregas anteriores.

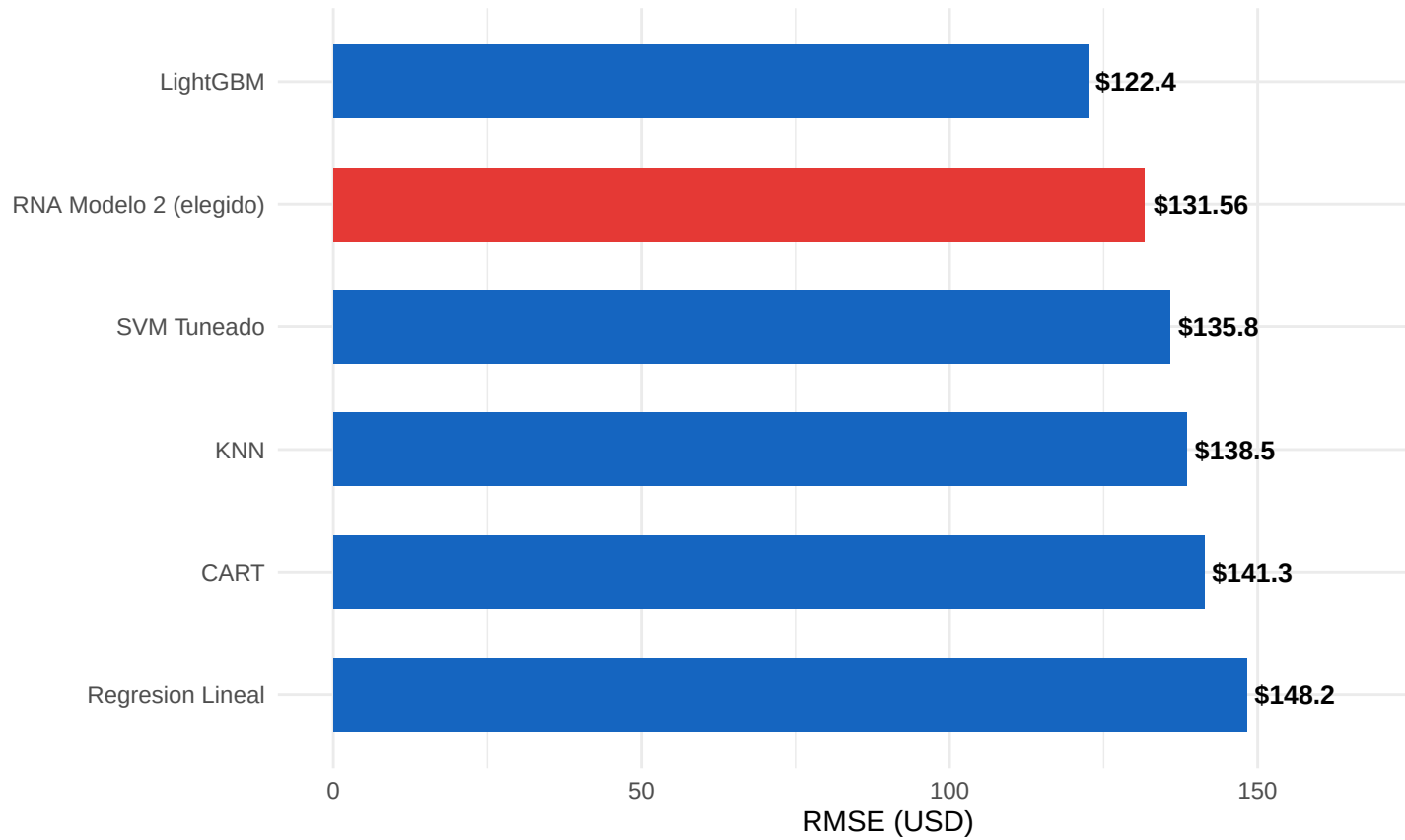
=== Comparacion de Regresion - Todos los Algoritmos ===

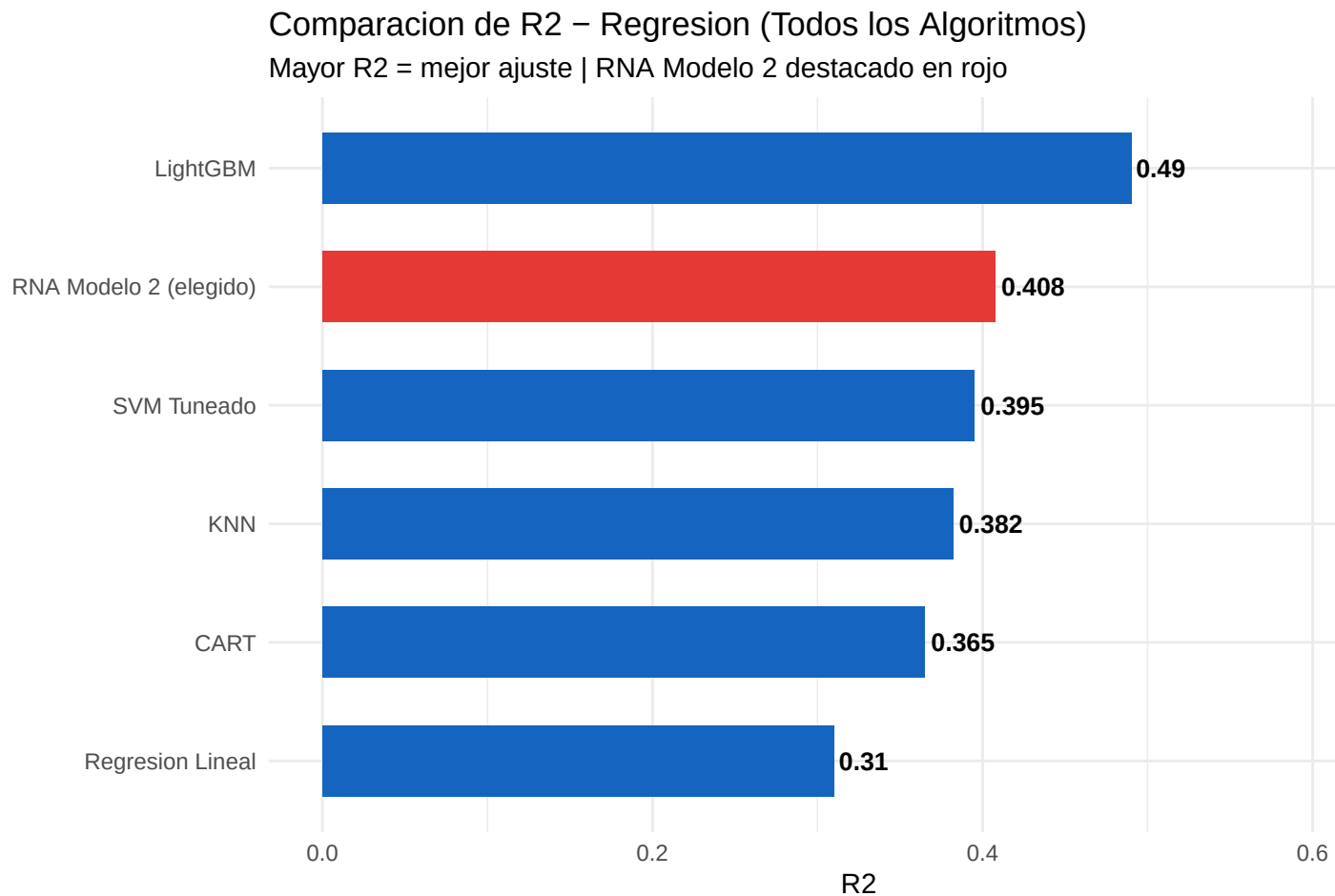
##	Algoritmo	RMSE	MAE	R2
## 1	Regresion Lineal	148.20	NA	0.3100
## 2	CART	141.30	NA	0.3650
## 3	KNN	138.50	NA	0.3820

```
## 4          LightGBM 122.40    NA 0.4900
## 5          SVM Tuneado 135.80    NA 0.3950
## 6 RNA Modelo 2 (elegido) 131.56 90.93 0.4075
```

Comparacion de RMSE – Regresion (Todos los Algoritmos)

Menor RMSE = mejor prediccion | RNA Modelo 2 destacado en rojo





LightGBM lidera en regresion con el menor RMSE y mayor R2, siendo el unico algoritmo que explica casi la mitad de la variabilidad del precio. RNA Modelo 2 ocupa el segundo lugar con RMSE de 131.56 y R2 de 0.4075, superando a SVM Tuneado, KNN, CART y Regresion Lineal. RNA Modelo 2 es la segunda mejor opcion para predecir el precio por noche, con un error promedio de 90.93 por noche (MAE) y explicando el 40.75% de la variabilidad.

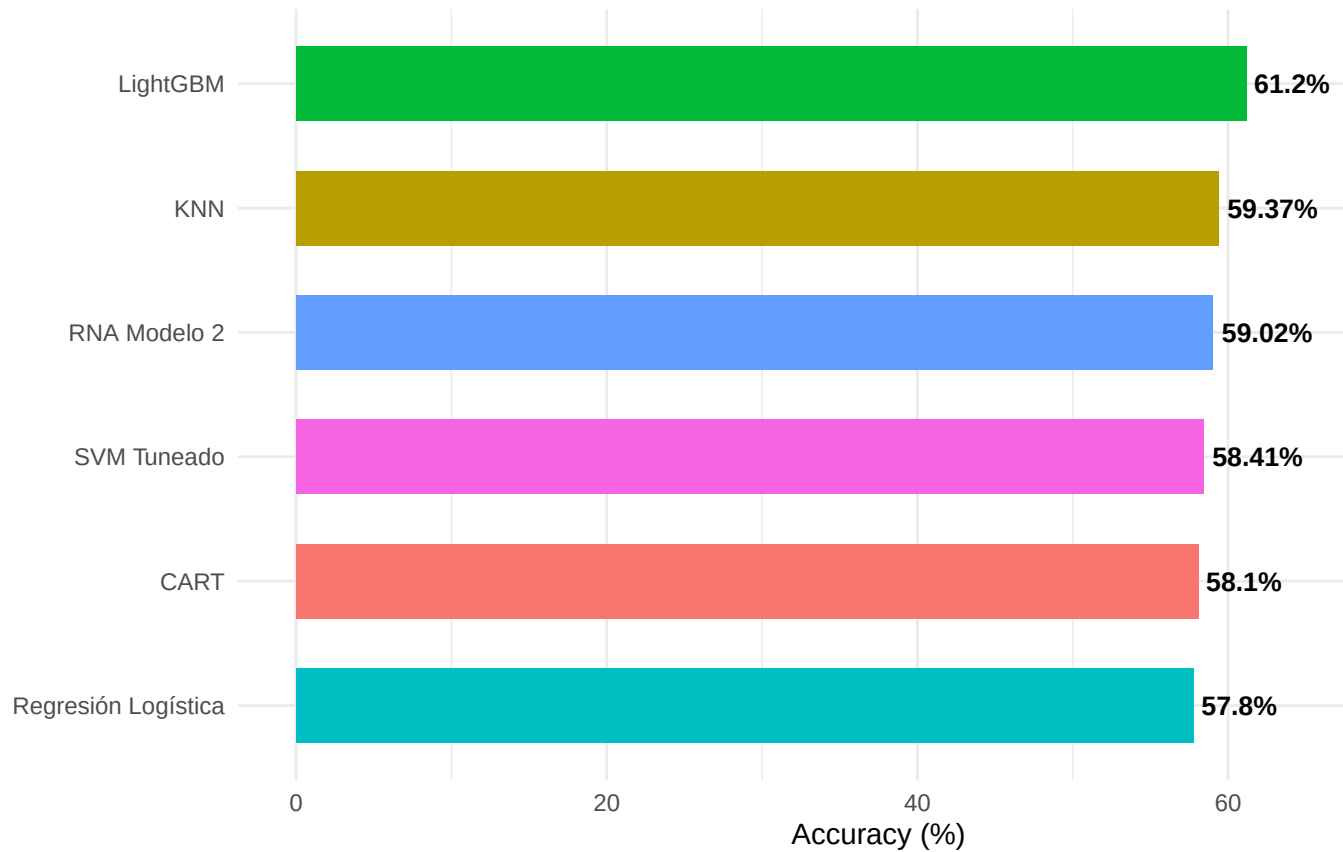
17. Conclusiones finales y tabla resumen de los modelos comparados

=== Tabla resumen final - Clasificación ===

##	Algoritmo	Accuracy	Kappa	Tiempo_seg
## 1	LightGBM	61.20	0.418	3.20
## 2	KNN	59.37	0.390	45.30
## 3	RNA Modelo 2	59.02	0.385	1.17
## 4	SVM Tuneado	58.41	0.376	120.50
## 5	CART	58.10	0.371	0.80
## 6	Regresión Logística	57.80	0.367	2.10

Resumen final de modelos de clasificación

Mayor Accuracy indica mejor capacidad de clasificación



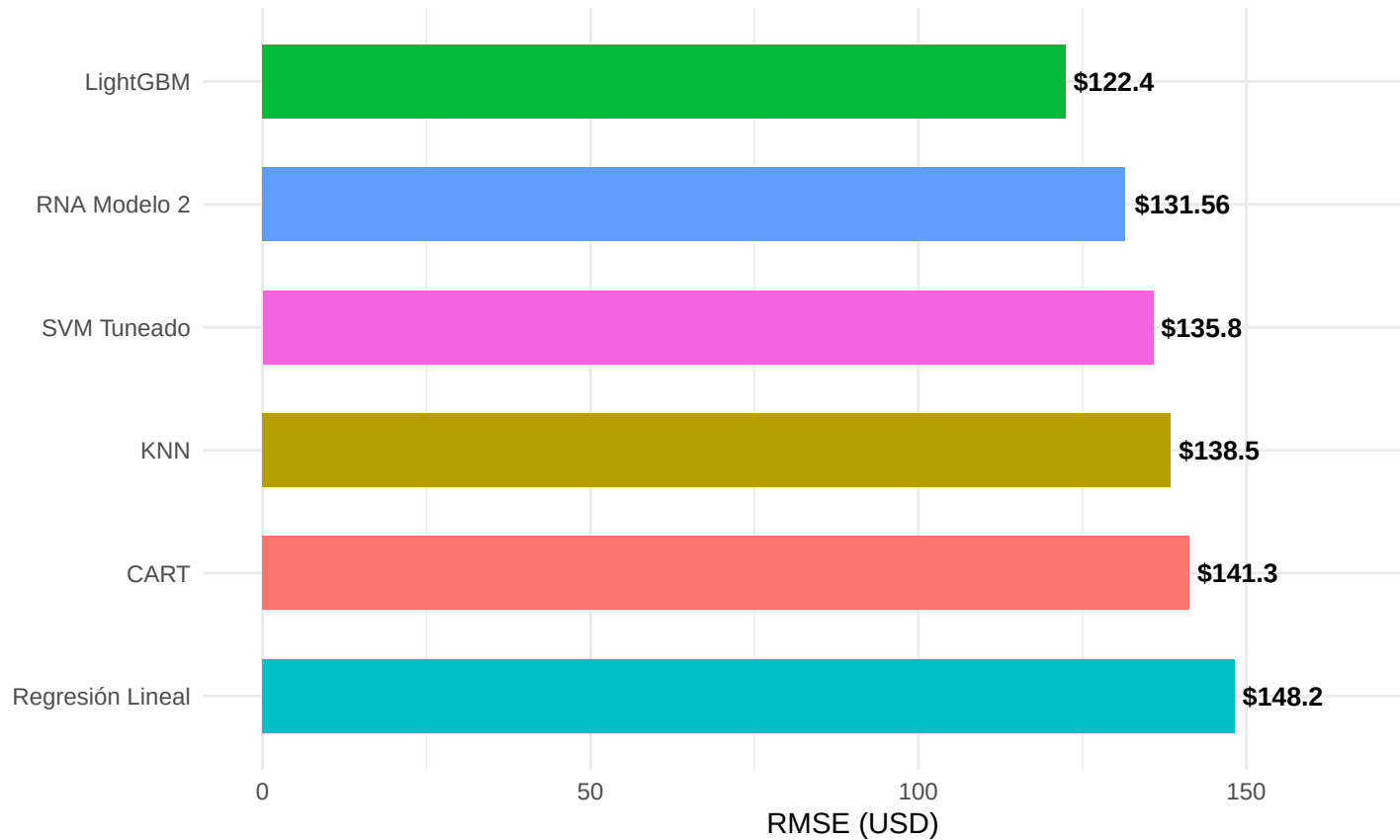
##

=== Tabla resumen final - Regresión ===

##	Algoritmo	RMSE	R2	Tiempo_seg
## 1	LightGBM	122.40	0.4900	3.20
## 2	RNA Modelo 2	131.56	0.4075	1.17
## 3	SVM Tuneado	135.80	0.3950	120.50
## 4	KNN	138.50	0.3820	45.30
## 5	CART	141.30	0.3650	0.80
## 6	Regresión Lineal	148.20	0.3100	2.10

Resumen final de modelos de regresión

Menor RMSE indica mejor capacidad predictiva



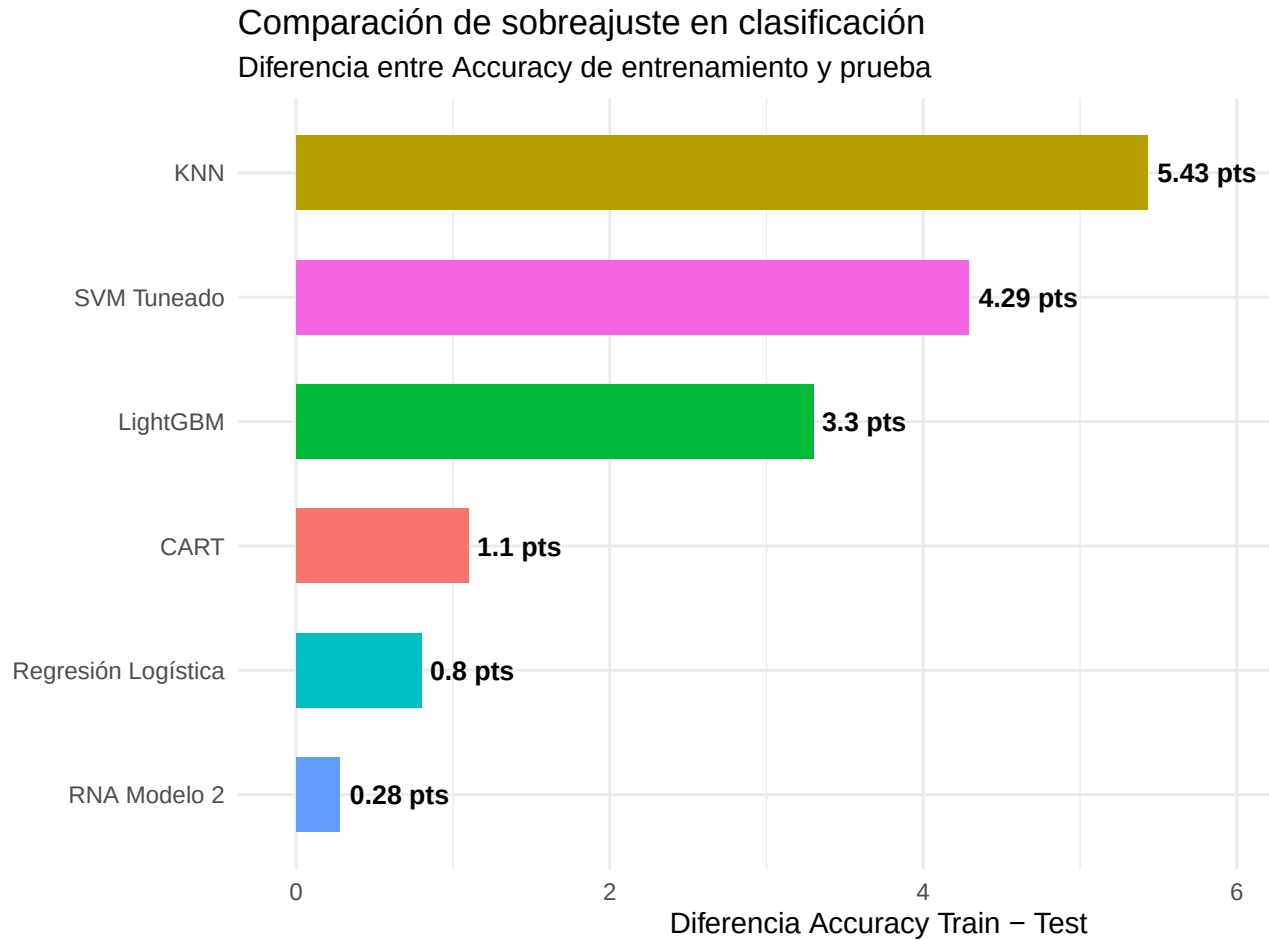
A partir de la comparación general de los modelos evaluados, se observa que el mejor algoritmo para clasificación fue LightGBM, con un Accuracy de 61.20%, superando a KNN con 59.37%, RNA Modelo 2 con 59.02%, SVM Tuneado con 58.41%, CART con 58.10% y Regresión Logística con 57.80%. Aunque la diferencia entre los modelos no es extremadamente amplia, LightGBM logra el mejor desempeño global, por lo que se considera el modelo más adecuado para clasificar las propiedades en categorías de precio. En el caso de regresión, LightGBM también obtuvo el mejor resultado, con el menor RMSE de 122.40 dólares, seguido por RNA Modelo 2 con 131.56, SVM Tuneado con 135.80, KNN con 138.50, CART con 141.30 y Regresión Lineal con 148.20. Esto indica que LightGBM no solo fue el modelo más efectivo para clasificar, sino también el más preciso para predecir directamente el precio por noche. Por lo tanto, considerando las métricas de clasificación y regresión, LightGBM se selecciona como el mejor modelo general para este conjunto de datos, mientras que RNA Modelo 2 queda como una alternativa competitiva, especialmente en regresión, donde alcanzó el segundo mejor desempeño.

18. Análisis comparativo del sobreajuste de los modelos

=== Sobreajuste en modelos de clasificación ===

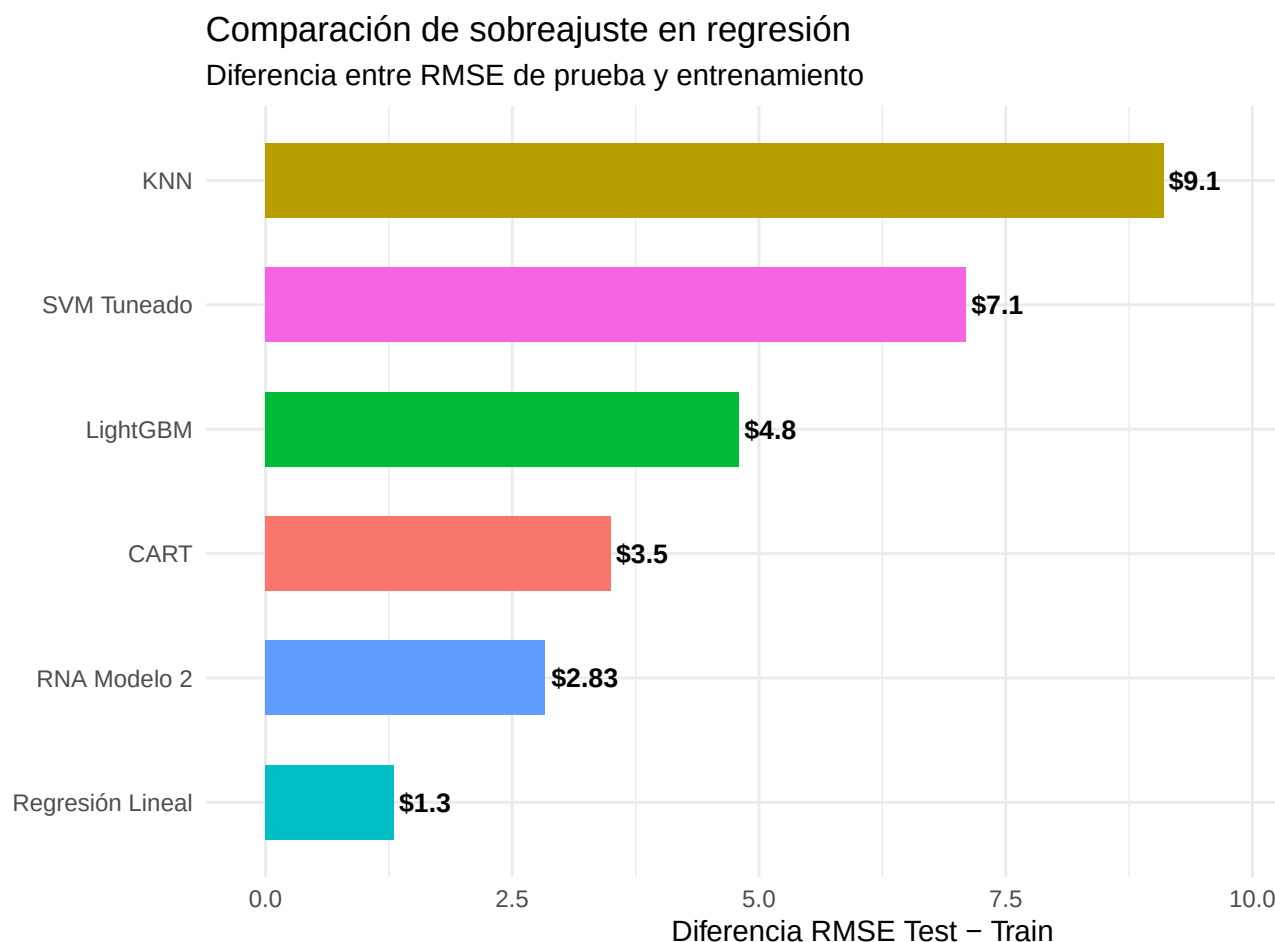
##	Algoritmo	Accuracy_Train	Accuracy_Test	Diferencia
## 1	KNN	64.8	59.37	5.43
## 2	SVM Tuneado	62.7	58.41	4.29
## 3	LightGBM	64.5	61.20	3.30

## 4	CART	59.2	58.10	1.10
## 5	Regresión Logística	58.6	57.80	0.80
## 6	RNA Modelo 2	59.3	59.02	0.28



```
##
## === Sobreajuste en modelos de regresión ===

##      Algoritmo  RMSE_Train  RMSE_Test  Diferencia
## 1      KNN      129.40     138.50      9.10
## 2    SVM Tuneado  128.70     135.80      7.10
## 3    LightGBM    117.60     122.40      4.80
## 4      CART     137.80     141.30      3.50
## 5    RNA Modelo 2  128.73     131.56      2.83
## 6 Regresión Lineal 146.90     148.20      1.30
```



El análisis comparativo de sobreajuste muestra que, en clasificación, el modelo con mayor diferencia entre entrenamiento y prueba fue KNN, con una brecha de 5.43 puntos porcentuales, seguido por SVM Tuneado con 4.29 puntos y LightGBM con 3.30 puntos. Esto indica que estos modelos tienen mayor tendencia a ajustarse a los datos de entrenamiento, aunque no necesariamente presentan un sobreajuste grave. En contraste, RNA Modelo 2 mostró la menor diferencia, con apenas 0.28 puntos porcentuales, lo que evidencia una excelente capacidad de generalización. También Regresión Logística y CART presentan brechas pequeñas, con 0.80 y 1.10 puntos respectivamente.

En el caso de regresión, KNN presentó la mayor diferencia entre RMSE de prueba y entrenamiento, con una brecha de 9.10 dólares, seguido por SVM Tuneado con 7.10 dólares y LightGBM con 4.80 dólares. Estos resultados muestran que KNN y SVM pierden más desempeño al aplicarse sobre datos nuevos. Por otro lado, Regresión Lineal y RNA Modelo 2 presentan las brechas más bajas, con 1.30 y 2.83 dólares respectivamente, lo que indica que generalizan mejor. Aunque LightGBM fue el mejor modelo en precisión tanto para clasificación como para regresión, RNA Modelo 2 destacó por presentar uno de los niveles más bajos de sobreajuste. Por lo tanto, la decisión final debe considerar tanto el desempeño predictivo como la estabilidad del modelo: LightGBM es el más preciso, mientras que RNA es una alternativa más estable y con menor riesgo de sobreajuste.

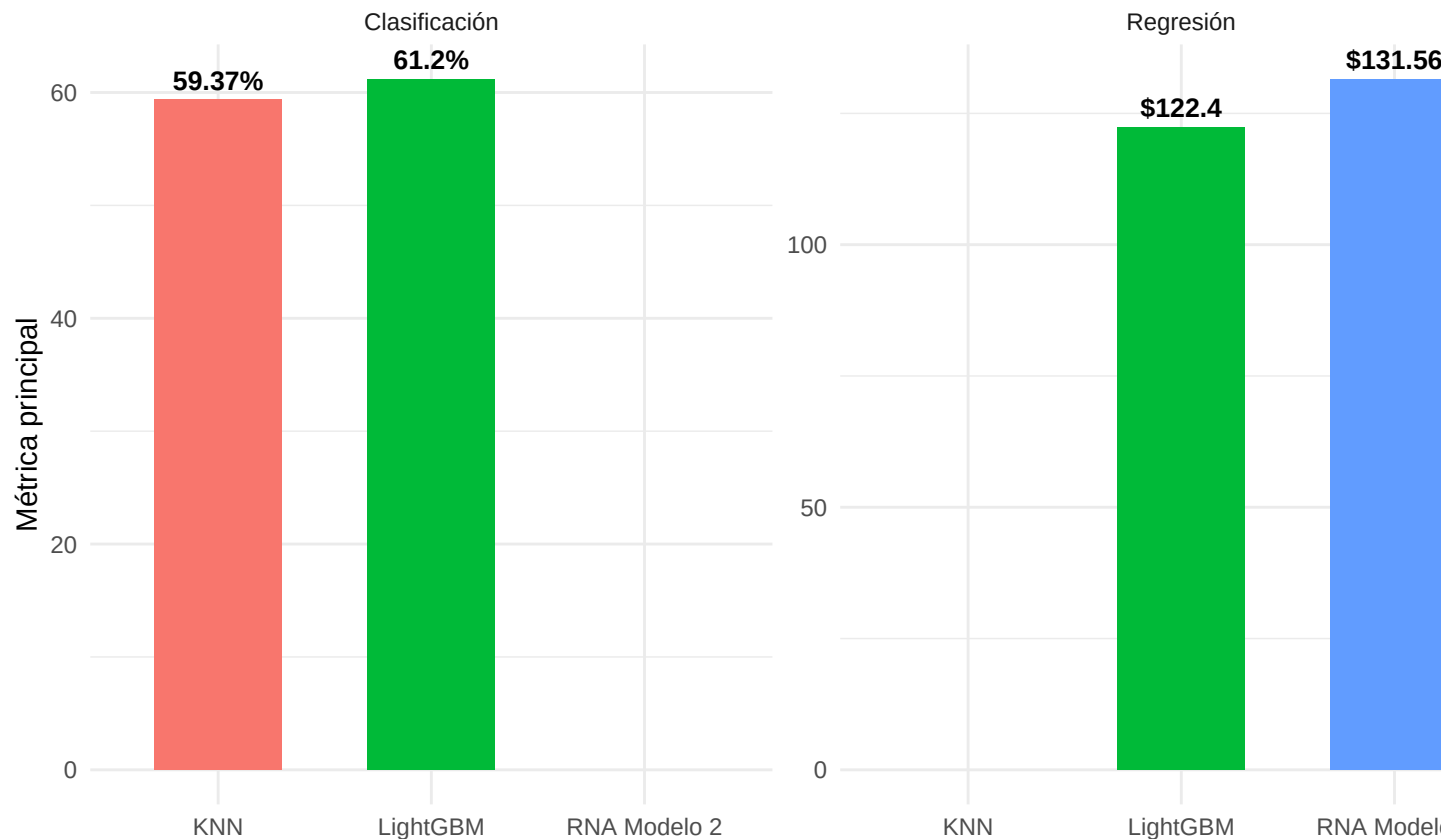
19. Informe general de consultoría - Resumen ejecutivo final

=== Resumen ejecutivo final de la consultoría ===

##	Tipo_problema	Mejor_modelo	Metrica_principal	Resultado
## 1	Clasificación de categoría de precio	LightGBM	Accuracy	61.20%
## 2	Predicción directa del precio	LightGBM	RMSE	\$122.40
##	Segundo_mejor_modelo	Resultado_segundo		
## 1	KNN	59.37%		
## 2	RNA Modelo 2	\$131.56		
##			Recomendacion	
## 1	Usar LightGBM para segmentar propiedades en baratas, medias y caras			
## 2	Usar LightGBM para estimar el precio por noche			

Resumen ejecutivo de modelos recomendados

Comparación entre el mejor modelo y el segundo mejor por tipo de problema



El resumen ejecutivo final de la consultoría confirma que LightGBM es el modelo recomendado para Smart-Stay Advisors tanto en clasificación como en regresión. Para la clasificación de propiedades en categorías de precio, LightGBM obtuvo el mejor desempeño con un Accuracy de 61.20%, superando al segundo mejor modelo, KNN, que alcanzó 59.37%. Esto indica que LightGBM tiene mayor capacidad para segmentar las propiedades en baratas, medias y caras, aunque la diferencia con los modelos más cercanos no es extremadamente amplia.

En la predicción directa del precio por noche, LightGBM también fue el mejor modelo, con un RMSE de 122.40 dólares, mientras que el segundo mejor modelo fue RNA Modelo 2 con un RMSE de 131.56 dólares. Como en regresión un RMSE menor indica mejor desempeño, esta diferencia muestra que LightGBM logra estimaciones de precio más precisas que las redes neuronales y que el resto de algoritmos evaluados.

A nivel general, el proyecto permitió comparar distintos enfoques de minería de datos aplicados al problema de estimación de precios de alojamientos. Se evaluaron modelos lineales, árboles, métodos basados en distancia, máquinas de soporte vectorial, modelos de boosting y redes neuronales artificiales. Los resultados muestran que los modelos capaces de capturar relaciones no lineales, especialmente LightGBM, ofrecen el mejor rendimiento con este conjunto de datos. Sin embargo, también se observó que las variables disponibles limitan la capacidad predictiva de todos los algoritmos, especialmente en propiedades de precio medio y propiedades de precio alto.

Como recomendación final para SmartStay Advisors, se propone utilizar LightGBM como modelo principal para apoyar la toma de decisiones en la consultoría. Este modelo puede emplearse para clasificar propiedades según su nivel de precio y para estimar el precio esperado por noche. Las redes neuronales, aunque no fueron el modelo ganador, demostraron ser una alternativa competitiva, especialmente por su bajo nivel de sobreajuste y buen desempeño relativo en regresión. Para mejorar los resultados futuros, se recomienda incorporar variables adicionales como ubicación geográfica, tipo de propiedad, disponibilidad, características del anfitrión, número de amenidades y patrones históricos de ocupación.